



Tecnun Universidad de Navarra

Proyecto Fin de Máster

ANÁLISIS DE DATOS EN INGENIERÍA

Influencia del codificador visual y su estrategia de entrenamiento en
Diffusion Policy para manipulación robótica: estudio en Push-T

Moises Britez

Director: Diego Borro

Donostia-San Sebastián, septiembre de 2026

Pº Manuel Lardizabal, 13. 20018 Donostia-San Sebastián, Gipuzkoa
Tel. 943 219 877 · Fax 943 311 442 · www.tecnun.es

Influencia del codificador visual y su estrategia de entrenamiento en Diffusion Policy para manipulación robótica: estudio en Push-T

Memoria presentada por Moises Britez para la obtención del título de Máster Universitario en análisis de datos en ingeniería.

Director: Diego Borro

Fdo.: Moises Britez

Fdo.: el director

Donostia-San Sebastián, septiembre de 2026

AGRADECIMIENTOS

[Texto de agradecimientos. Esta pagina es opcional y no lleva numeracion de capitulo. Se redacta en primera persona, unica excepcion al estilo impersonal del resto de la memoria.]

ÍNDICE DE CONTENIDOS

Agradecimientos	iii
Resumen del proyecto	xi
Abstract	xiii
1. Introducción y objetivos	1
1.1 Contexto y motivación	1
1.2 Problema abordado	1
1.3 Objetivos	2
1.4 Alcance del trabajo	2
1.5 Estructura de la memoria	2
2. Estado del arte	3
2.1 Aprendizaje por imitación en manipulación robótica	3
2.2 Fundamentos de los modelos de difusión	3
2.3 Diffusion Policy para control visuomotor	4
2.4 Evolución de las políticas de difusión	5
2.5 Codificadores visuales y preentrenamiento	6
2.5.1 Redes residuales y supervisión en ImageNet	6
2.5.2 Transformadores visuales y aprendizaje autosupervisado	7
2.5.3 Preentrenamiento multimodal con CLIP	7
2.5.4 Representaciones visuales preentrenadas para control	7
2.5.5 Estrategias de adaptación	8
2.6 Síntesis y hueco de investigación	8
3. Metodología	11
3.1 Planteamiento experimental	11
3.2 Tarea y conjunto de datos	11
3.3 Arquitectura de la política	12
3.4 Variantes del codificador visual	13
3.5 Protocolo de entrenamiento	14
3.6 Protocolo de evaluación	16
3.7 Métricas	16
3.8 Entorno de ejecución y reproducibilidad	17
3.9 Coste temporal de los entrenamientos	19
4. Resultados	21
4.1 Cobertura de las ejecuciones	21
4.2 Evolución del rendimiento	21

4.3	Ajuste y generalización	23
4.4	Coste computacional	24
4.5	Comprobación cualitativa	26
4.6	Contraste de las hipótesis	26
4.7	Limitaciones	27
5.	Conclusiones	29
5.1	Conclusiones del trabajo	29
5.2	Trabajo futuro	31
6.	Presupuesto	33
6.1	Material fungible y suministros	33
6.2	Amortización de los equipos	33
6.3	Licencias de software	34
6.4	Mano de obra	34
6.5	Resumen del presupuesto	34
	Referencias	37
	Bibliografía	41
	A. Título del primer anexo	43
	B. Título del segundo anexo	45

ÍNDICE DE FIGURAS

Figura 1.	Evolución de la puntuación media de evaluación de las cinco variantes.	22
Figura 2.	Evolución de las pérdidas de entrenamiento y validación.	23
Figura 3.	Inferencia de V1 y V2 sobre una condición inicial compartida.	27

ÍNDICE DE TABLAS

Tabla 1.	Resultado de la ablación de codificadores visuales de <i>Diffusion Policy</i> en la tarea <i>Square</i>	5
Tabla 2.	Principales líneas de extensión de <i>Diffusion Policy</i>	6
Tabla 3.	Configuración de las cinco variantes del codificador visual evaluadas.	13
Tabla 4.	Hiperparámetros de entrenamiento de las cinco variantes.	15
Tabla 5.	Plataforma de ejecución de los experimentos.	18
Tabla 6.	Componentes de la pila de software y función de cada uno.	18
Tabla 7.	Coste temporal de los entrenamientos ejecutados.	19
Tabla 8.	Cobertura de los entrenamientos y mejor puntuación media de evaluación.	21
Tabla 9.	Diferencias pareadas entre los puntos de control seleccionados. . .	23
Tabla 10.	Latencia de inferencia de las cinco variantes, en milisegundos. . . .	24
Tabla 11.	Pico de memoria gráfica reservada, en gibibytes.	25
Tabla 12.	Presupuesto de material fungible y suministros.	33
Tabla 13.	Amortización de los equipos imputada al proyecto.	34
Tabla 14.	Dedicación del autor por fases del proyecto.	35
Tabla 15.	Presupuesto de mano de obra.	35
Tabla 16.	Resumen del presupuesto.	35

RESUMEN DEL PROYECTO

Diffusion Policy genera acciones de manipulación robótica mediante un proceso de difusión condicionado por la observación. Su variante visuomotora encadena un codificador visual y una red generativa, de modo que la representación de la imagen delimita la información disponible para el control. La investigación posterior ha modificado sobre todo el bloque generativo, mientras que la elección del codificador y de su estrategia de entrenamiento sigue sin resolverse cuando solo se dispone de un conjunto reducido de demostraciones.

Este trabajo evalúa la influencia del codificador visual y de su estrategia de entrenamiento sobre el rendimiento de una *Diffusion Policy* en la tarea Push-T. Con ese fin se implementan cinco variantes del codificador dentro de la misma política: una ResNet-18 entrenada desde cero, la misma red preentrenada sobre ImageNet en versiones congelada y con ajuste fino, y dos transformadores de visión congelados, DINOv2 ViT-S/14 y CLIP ViT-B/16. Todas entregan un vector de 512 componentes y comparten red generativa, datos y protocolo de evaluación. El ajuste emplea 206 demostraciones teleoperadas y presupuestos de 200 a 500 épocas, con parada anticipada. La evaluación mide la puntuación media de cobertura sobre 50 episodios generados con semillas reservadas, junto con el tiempo de cómputo consumido.

La línea base entrenada desde cero alcanza 0,864. Las cuatro variantes preentrenadas obtienen 0,668, 0,648, 0,622 y 0,535. V0 supera a todas con significación tras corregir los diez contrastes por pares mediante el procedimiento de Holm. En cambio, ninguna diferencia entre variantes preentrenadas supera el umbral familiar del 5 %. Cada época de V0 ocupa 1,6 min, frente a un intervalo de 2,3 min a 14,7 min en las demás. Las cinco ejecuciones suman 237,1 h.

Se concluye que, en Push-T y con pocas demostraciones, ninguno de los bloques preentrenados evaluados mejora el rendimiento ni el coste de la línea base. Esta ventaja debe atribuirse al codificador junto con su preprocesado, puesto que ambos varían de forma conjunta.

Palabras clave: aprendizaje por imitación, modelos de difusión, codificador visual, transferencia de aprendizaje, manipulación robótica.

ABSTRACT

Diffusion Policy generates robotic manipulation actions through a diffusion process conditioned on the observation. Its visuomotor variant chains a visual encoder and a generative network, so the image representation bounds the information available for control. Subsequent research has mostly modified the generative block, whereas the choice of encoder and of its training strategy remains unsettled when only a small set of demonstrations is available.

This work evaluates the influence of the visual encoder and of its training strategy on the performance of a *Diffusion Policy* on the Push-T task. To that end, five encoder variants are implemented within the same policy: a ResNet-18 trained from scratch, the same network pretrained on ImageNet in frozen and fine-tuned versions, and two frozen vision transformers, DINOv2 ViT-S/14 and CLIP ViT-B/16. All of them output a 512-component vector and share the generative network, the data, and the evaluation protocol. Training uses 206 teleoperated demonstrations and budgets ranging from 200 to 500 epochs, with early stopping. Evaluation measures the mean coverage score over 50 episodes generated with held-out seeds, together with the compute time consumed.

The baseline trained from scratch reaches 0.864. The four pretrained variants score 0.668, 0.648, 0.622, and 0.535. V0 outperforms all of them after the ten pairwise tests are corrected with Holm’s procedure. By contrast, no difference among the pretrained variants exceeds the family-wise 5 % threshold. Each V0 epoch takes 1.6 min, compared with 2.3 min to 14.7 min for the other variants. The five runs total 237.1 h.

The study concludes that, on Push-T and with few demonstrations, none of the evaluated pretrained visual blocks improves the baseline in performance or cost. This advantage must be ascribed to the encoder together with its preprocessing pipeline, since both vary jointly.

Keywords: imitation learning, diffusion models, visual encoder, transfer learning, robotic manipulation.

1. INTRODUCCIÓN Y OBJETIVOS

Este capítulo sitúa el trabajo en su ámbito, delimita el problema que aborda y enuncia los objetivos que guían la investigación. Además, precisa el alcance del estudio y describe la organización de la memoria.

1.1 Contexto y motivación

La manipulación robótica requiere generar acciones continuas a partir de observaciones sensoriales de alta dimensión. El aprendizaje por imitación aborda esa dificultad ajustando una política sobre demostraciones humanas, sin necesidad de diseñar una función de recompensa [1].

Sin embargo, las demostraciones humanas son multimodales. Ante una misma observación, un operario puede ejecutar acciones distintas y todas ellas válidas. Los métodos de clonación de comportamiento basados en regresión promedian esas alternativas y producen trayectorias incoherentes [2].

Para evitar ese promediado, Chi *et al.* [3] proponen *Diffusion Policy*, que representa la acción como un proceso de difusión condicionado por la observación. El método hereda así la capacidad de los modelos generativos de difusión para representar distribuciones multimodales [4].

En su variante visuomotora, *Diffusion Policy* se compone de dos bloques. Un codificador visual transforma las imágenes en un vector de características, y una red generativa produce la secuencia de acciones condicionada por dicho vector. Por tanto, la representación visual determina qué información alcanza al bloque generativo.

1.2 Problema abordado

La investigación sobre *Diffusion Policy* ha modificado principalmente el bloque generativo y la modalidad de entrada. Algunos trabajos sustituyen las imágenes por nubes de puntos [5], mientras que otros reducen el coste de inferencia [6].

El trabajo original incluye una ablación de ResNet y CLIP en la tarea *Square* [3]. Además, un estudio posterior compara ResNet-18 y DINOv3 en cuatro tareas, incluida Push-T [7]. Sin embargo, ninguno contrasta conjuntamente ResNet-18, DINOv2 y CLIP bajo el mismo protocolo de Push-T.

Por tanto, persisten dos cuestiones para esa configuración experimental. La primera afecta a la arquitectura, puesto que no se ha establecido qué codificador visual conviene con un conjunto reducido de demostraciones. La segunda afecta a la estrategia de entrenamiento: desde cero, congelación o ajuste fino. Responder a ambas exige aislar el codificador visual y mantener constante el resto del sistema.

1.3 Objetivos

El objetivo general consiste en evaluar la influencia del codificador visual y de su estrategia de entrenamiento sobre el rendimiento de una *Diffusion Policy* aplicada a la tarea Push-T.

Este propósito se concreta en los objetivos específicos siguientes:

1. Implementar cinco variantes del codificador visual dentro de la misma *Diffusion Policy*, manteniendo constantes la arquitectura de control y la configuración experimental.
2. Comparar el rendimiento de las variantes mediante la puntuación media obtenida en los entornos de evaluación de Push-T.
3. Analizar el coste computacional de cada variante a partir de sus parámetros entrenables, sus tiempos de ejecución y su consumo de memoria gráfica.

1.4 Alcance del trabajo

El estudio se circunscribe a la tarea Push-T en simulación, que consiste en desplazar una pieza con forma de T hasta una posición objetivo mediante un efector circular [3]. Todas las variantes comparten la misma política generativa, el conjunto de datos y el protocolo de evaluación. La variable manipulada es el bloque visual junto con su estrategia de entrenamiento; las desviaciones de presupuesto y tamaño de lote se documentan de forma explícita.

Quedan fuera del alcance tres aspectos. En primer lugar, la validación sobre un robot físico, puesto que el trabajo opera únicamente en simulación. En segundo lugar, el análisis de robustez frente a perturbaciones visuales. En tercer lugar, el contraste estadístico entre semillas de entrenamiento, limitado por el coste computacional disponible.

1.5 Estructura de la memoria

La memoria se organiza en seis capítulos. El Capítulo 2 revisa los fundamentos de los modelos de difusión, su aplicación al control visuomotor y las familias de codificadores visuales consideradas. El Capítulo 3 describe el diseño experimental, el conjunto de datos, las variantes evaluadas y las métricas.

A continuación, el Capítulo 4 presenta las curvas de entrenamiento, la comparativa de rendimiento y el coste computacional de cada variante. El Capítulo 5 interpreta esos resultados, señala las limitaciones del estudio y propone líneas de continuación. Por último, el Capítulo 6 detalla los costes del proyecto.

2. ESTADO DEL ARTE

Este capítulo revisa los fundamentos que permiten estudiar la influencia del codificador visual en una política de difusión. La exposición parte del aprendizaje por imitación y continúa con los modelos generativos de difusión. Después, se describe *Diffusion Policy* y se analizan sus principales extensiones.

A continuación, la revisión se centra en las representaciones visuales empleadas en este trabajo. Se consideran las redes residuales, los transformadores visuales y el preentrenamiento supervisado, autosupervisado y multimodal. El capítulo concluye delimitando el hueco experimental que aborda la metodología.

2.1 Aprendizaje por imitación en manipulación robótica

El aprendizaje por imitación obtiene una política a partir de demostraciones de un experto. Su formulación más directa, denominada clonación del comportamiento, trata cada par observación-acción como un ejemplo supervisado. Este planteamiento evita diseñar una función de recompensa, pero depende de la cobertura y calidad de las demostraciones [1], [8].

Sin embargo, el entrenamiento supervisado y la ejecución de la política siguen distribuciones diferentes. Un error modifica el estado visitado y puede llevar al robot fuera de los datos de entrenamiento. Los errores posteriores se acumulan porque la política carece de ejemplos en esas nuevas regiones. DAgger reduce este desplazamiento mediante la agregación iterativa de correcciones del experto [1].

Además, las demostraciones de manipulación suelen contener varias acciones válidas para una misma observación. Una pérdida cuadrática aplicada a una política determinista aproxima la media condicional de esas alternativas. Dicha media puede situarse entre modos válidos y generar una acción que no aparece en ninguna demostración [2].

Para representar esa multimodalidad, la clonación de comportamiento implícita define una función de energía sobre pares de observación y acción. La acción se obtiene minimizando esa energía durante la inferencia. Este enfoque mejora la expresividad, aunque requiere muestras negativas y una optimización adicional [2]. Estas limitaciones motivan el uso de modelos generativos directamente sobre las secuencias de control.

2.2 Fundamentos de los modelos de difusión

Los modelos de difusión aprenden a invertir una degradación gradual de los datos. Los modelos probabilísticos de difusión por eliminación de ruido (DDPM) definen un proceso directo que añade ruido gaussiano durante K pasos. Para una muestra limpia $\mathbf{x}^0$, un estado perturbado puede expresarse como [4]

$$q(\mathbf{x}^k | \mathbf{x}^0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_k} \mathbf{x}^0, (1 - \bar{\alpha}_k)\mathbf{I}\right). \quad (1)$$

En cambio, el proceso inverso parte de ruido y estima sucesivamente una muestra de la distribución original. Ho *et al.* [4] parametrizan una red que predice el ruido incorporado en cada paso. Su objetivo simplificado es un error cuadrático entre el ruido real y el estimado, lo que permite un entrenamiento estable.

Asimismo, los modelos basados en puntuación aprenden la dirección de mayor crecimiento de la densidad logarítmica. Este campo permite generar muestras mediante dinámica de Langevin [9]. La introducción de varios niveles de ruido facilita la estimación en regiones con pocos datos.

Por otra parte, las ecuaciones diferenciales estocásticas (SDE) ofrecen una formulación continua común para estos procesos. El proceso inverso depende de la función de puntuación de las distribuciones perturbadas. Esta perspectiva conecta los DDPM con otros modelos basados en puntuación y amplía las alternativas de muestreo [10].

No obstante, el muestreo iterativo aumenta la latencia. Los modelos implícitos de difusión por eliminación de ruido (DDIM) conservan el objetivo de entrenamiento de los DDPM, pero emplean un proceso inverso no markoviano. Esta modificación reduce el número de evaluaciones necesarias durante la generación [11].

En conjunto, estos trabajos proporcionan tres elementos para el control robótico: una distribución generativa expresiva, un objetivo de predicción de ruido y un procedimiento iterativo de muestreo. *Diffusion Policy* traslada esos elementos desde la generación de datos al espacio de acciones.

2.3 Diffusion Policy para control visuomotor

Diffusion Policy modela la distribución condicional $p(\mathbf{A}_t \mid \mathbf{O}_t)$, donde $\mathbf{O}_t$ contiene las observaciones recientes y $\mathbf{A}_t$ representa una secuencia de acciones futuras [3]. Durante el entrenamiento, se perturba una secuencia demostrada y se minimiza la pérdida

$$\mathcal{L}_{DP} = \mathbb{E}_{k, \epsilon} \left[\|\epsilon - \epsilon_{\theta}(\mathbf{O}_t, \mathbf{A}_t^k, k)\|_2^2 \right]. \quad (2)$$

De este modo, la red ϵ_{θ} aprende a eliminar el ruido de una trayectoria condicionada por la observación. La formulación admite varios modos de acción sin promediarlos en una única salida. Además, evita estimar la constante de normalización que dificulta el entrenamiento de los modelos de energía [3].

En cuanto a la arquitectura, el trabajo original propone dos redes para predecir el ruido. La primera es una red convolucional temporal con estructura U-Net y modulación lineal por características (FiLM). La segunda emplea un transformador temporal con atención cruzada. La variante convolucional ofrece mayor estabilidad, mientras que el transformador responde mejor ante cambios de acción rápidos [3].

Además, la política aplica control de horizonte recesivo. A partir de una ventana de observaciones, predice una secuencia completa y ejecuta solo sus primeras acciones. Después, incorpora una nueva observación y repite la predicción. Esta estrategia combina coherencia temporal con capacidad de reacción ante desviaciones [3].

En la rama visual, cada imagen se transforma en un vector antes del proceso de eliminación de ruido. La implementación de referencia utiliza una ResNet-18 sin preentrenamiento. El

promedio espacial global se sustituye por un *spatial softmax* para conservar información posicional. Asimismo, la normalización por lotes se reemplaza por normalización por grupos para estabilizar el entrenamiento [3].

La influencia de esta rama ya se examina parcialmente en el artículo original. La Tabla 1 resume la comparación realizada sobre la tarea *Square* de RoboMimic. Se mantienen fijos el conjunto de demostraciones y la política convolucional, pero cambian la arquitectura y la estrategia de adaptación del codificador.

Tabla 1. Resultado de la ablación de codificadores visuales de *Diffusion Policy* en la tarea *Square*.

Codificador y preentrenamiento	Desde cero	Congelado	Ajuste fino
ResNet-18, ImageNet-21k	0,94	0,58	0,92
ResNet-34, ImageNet-21k	0,92	0,40	0,94
ViT-B/16, CLIP	0,22	0,70	0,98

Fuente: adaptado de [3, Tabla 5].

Como muestra la tabla, ninguna estrategia domina para todas las arquitecturas. El ajuste fino obtiene el mejor resultado con ViT-B/16, mientras que ResNet-18 desde cero supera ligeramente a su versión ajustada. En cambio, la congelación reduce el rendimiento de ambas redes residuales [3].

Por último, Push-T permite estudiar estas decisiones en una tarea de contacto planar. Un efector circular debe empujar una pieza con forma de T hasta una región objetivo. La puntuación depende del solapamiento entre la pieza y dicha región, por lo que exige controlar posición y orientación [2], [3]. En la versión física, la política entrenada de extremo a extremo alcanzó un 0,95 de éxito, frente a 0,80 con R3M congelado y 0,15 con ImageNet congelado [3]. Estos resultados confirman que la representación visual puede alterar el control aun cuando la red generativa permanece fija.

2.4 Evolución de las políticas de difusión

Tras la publicación de *Diffusion Policy*, la investigación se ha dividido en varias líneas. Unos trabajos modifican la representación sensorial, mientras que otros actúan sobre la jerarquía, la geometría o la velocidad de inferencia. La Tabla 2 sintetiza las extensiones más relacionadas con este estudio.

En primer lugar, DP3, HDP y ET-SEED incorporan estructura geométrica mediante nubes de puntos o restricciones cinemáticas [5], [12], [14]. Estas propuestas mejoran la generalización espacial, pero cambian simultáneamente la modalidad y la arquitectura perceptiva. Por ello, sus resultados no permiten aislar el efecto de un codificador de imagen 2D.

En segundo lugar, *Consistency Policy* reduce el coste del muestreo mediante destilación. El trabajo mantiene constante la codificación de imagen y concentra la comparación en la etapa generativa [6]. De forma análoga, la percepción activa modifica el espacio de control para coordinar la cámara y el manipulador [13].

Finalmente, el estudio DINOv3-DP es el antecedente más próximo a esta investigación. Compara ResNet-18 y un ViT-S/16 autosupervisado en Push-T, Lift, Can y Square. Además, evalúa entrenamiento desde cero, congelación y ajuste fino durante 100 épocas [7]. Sus

Tabla 2. Principales líneas de extensión de *Diffusion Policy*.

Trabajo	Componente modificado	Representación	Contribución principal
DP3 [5]	Percepción	Nube de puntos	Sustituye la imagen 2D por una representación geométrica 3D compacta.
HDP [12]	Jerarquía y cinemática	RGB-D y nube de puntos	Separa la planificación de alto nivel del control difusivo local.
Consistency Policy [6]	Inferencia	Imagen 2D	Destila la trayectoria de eliminación de ruido para reducir la latencia.
Percepción activa [13]	Espacio de estado y acción	Cámara móvil RGB	Coordina el punto de vista mediante una restricción cinemática.
ET-SEED [14]	Equivarianza geométrica	Nube de puntos	Introduce simetría SE(3) en la generación de trayectorias.
DINOv3-DP [7]	Codificador visual	Imagen 2D	Compara ResNet-18 y DINOv3 bajo tres estrategias de entrenamiento.

Fuente: elaboración propia.

resultados muestran que la ventaja del preentrenamiento depende de la tarea y de la estrategia de adaptación.

2.5 Codificadores visuales y preentrenamiento

La revisión anterior muestra que la imagen condiciona toda la secuencia de acciones. Por tanto, el codificador debe conservar la geometría relevante y descartar variaciones que no afecten a la tarea. Las familias seleccionadas difieren tanto en su arquitectura como en la señal utilizada durante el preentrenamiento.

2.5.1 Redes residuales y supervisión en ImageNet

Las redes residuales aprenden funciones de corrección respecto a la entrada de cada bloque. Las conexiones de identidad facilitan la optimización de redes profundas y permiten reutilizar características en distintas escalas [15]. En particular, ResNet-18 ofrece un compromiso entre capacidad y coste que favorece su uso en políticas visuomotoras.

Asimismo, el preentrenamiento supervisado en ImageNet emplea etiquetas de objeto sobre un conjunto visual amplio [16]. Estos pesos proporcionan detectores generales de textura, forma y categoría. Sin embargo, la clasificación no exige conservar toda la posición espacial que requiere una tarea de contacto.

Por esta razón, una representación adecuada para reconocimiento puede no serlo para control.

2.5.2 Transformadores visuales y aprendizaje autosupervisado

En cambio, el transformador visual (ViT) divide la imagen en parches y los procesa como una secuencia de *tokens*. La autoatención relaciona regiones alejadas sin depender exclusivamente de la localidad convolucional. Su rendimiento mejora cuando existe un preentrenamiento a gran escala [17].

Sobre esta arquitectura, DINOv2 aprende mediante autodestilación sin etiquetas. El método combina datos visuales diversos y curados con una estrategia de entrenamiento autosupervisado. Sus características transfieren a tareas de imagen y de nivel de píxel sin ajuste específico [18].

En consecuencia, DINOv2 resulta pertinente para manipulación con pocas demostraciones. Sus representaciones densas pueden aportar correspondencias espaciales aprendidas fuera del dominio robótico. Sin embargo, la congelación impide adaptar esas características a la dinámica y a los contactos de Push-T. La comparación con DINOv3 confirma que esta decisión debe evaluarse para cada tarea [7].

2.5.3 Preentrenamiento multimodal con CLIP

Por su parte, el preentrenamiento contrastivo de imagen y lenguaje (CLIP) alinea una imagen con su descripción textual. El modelo original utiliza 400 millones de pares obtenidos de Internet y transfiere sus representaciones a distintas tareas visuales [19]. La supervisión lingüística favorece características con contenido semántico general.

Sin embargo, Push-T no requiere interpretar instrucciones ni reconocer categorías abiertas. La política necesita localizar una pieza, estimar su orientación y relacionarla con una meta. Por tanto, la semántica de CLIP solo resultará útil si su representación conserva también la información geométrica necesaria.

Además, la ablación original muestra una diferencia marcada entre congelación y ajuste fino. El ViT-B/16 preentrenado con CLIP obtiene 0,70 cuando permanece fijo y 0,98 cuando se adapta [3]. Este resultado impide suponer que un corpus de preentrenamiento mayor garantiza por sí solo una política mejor.

2.5.4 Representaciones visuales preentrenadas para control

Frente al preentrenamiento visual genérico, otra línea utiliza datos y objetivos relacionados con la interacción. R3M aprende de vídeo egocéntrico mediante objetivos temporales y alineación con lenguaje. Como módulo congelado, mejora varias tareas de manipulación con pocos datos [20]. No obstante, en Push-T queda por debajo del codificador entrenado de extremo a extremo [3].

Asimismo, MVP preentrena un ViT mediante modelado enmascarado sobre imágenes de interacción humana. Después, mantiene fijo el codificador y aprende controladores mediante aprendizaje por refuerzo. En las ocho tareas de PixMC, esta estrategia supera al preentrenamiento supervisado en ImageNet en siete casos [21]. Sin embargo, el estudio no utiliza demostraciones ni una política de difusión.

Por su parte, Voltron combina reconstrucción visual enmascarada con supervisión lingüística. Su evaluación sobre cinco problemas robóticos muestra que las representaciones reconstructivas favorecen detalles espaciales, mientras que los objetivos contrastivos capturan

mejor la semántica. El equilibrio entre ambos niveles depende de la tarea, y una mayor carga semántica puede perjudicar algunos problemas de control de bajo nivel [22].

Finalmente, CortexBench compara CLIP, R3M, MVP y VIP sobre 17 tareas de siete bancos de pruebas. Ninguna representación obtiene el mejor resultado en todos los dominios. VC-1 alcanza el mejor promedio, pero tampoco domina cada tarea; además, su adaptación específica produce mejoras sustanciales [23]. Por tanto, la escala del preentrenamiento no determina por sí sola la utilidad de un codificador para control.

2.5.5 Estrategias de adaptación

En términos generales, el entrenamiento desde cero adapta toda la representación a la tarea, pero depende exclusivamente de las demostraciones disponibles. Esta opción reduce el sesgo entre preentrenamiento y control a costa de una mayor necesidad de datos.

En cambio, un codificador congelado conserva el conocimiento previo y reduce los parámetros entrenables. También evita que un conjunto pequeño degrade las características iniciales. Su principal limitación es que la política solo puede adaptar las capas posteriores a una representación fija.

Por último, el ajuste fino permite adaptar los pesos preentrenados. Esta estrategia ofrece mayor flexibilidad, aunque incrementa la memoria necesaria y puede producir sobreajuste. La evidencia disponible no establece una regla universal: el resultado cambia con la arquitectura, la tarea y el dominio de preentrenamiento [3], [7].

2.6 Síntesis y hueco de investigación

En conjunto, la literatura confirma que las políticas de difusión representan acciones multimodales y mantienen coherencia temporal. Los trabajos posteriores han mejorado la geometría, la jerarquía y la velocidad de inferencia. Sin embargo, la representación visual 2D suele permanecer fija o cambia junto con otros componentes.

De forma complementaria, los estudios sobre representaciones preentrenadas para control descartan la existencia de un codificador universalmente superior. El resultado depende del dominio, del objetivo de preentrenamiento y de la estrategia de adaptación [21], [22], [23].

El artículo original constituye una excepción, puesto que compara ResNet-18, ResNet-34 y CLIP ViT-B/16 bajo tres estrategias. No obstante, esa ablación se limita a la tarea *Square* [3]. En Push-T físico solo se contrastan el entrenamiento de extremo a extremo y dos codificadores congelados, ImageNet y R3M.

Asimismo, DINOv3-DP amplía la evidencia a cuatro tareas e incluye Push-T. Ese estudio no evalúa DINOv2 ni incorpora CLIP en la misma matriz comparativa [7]. Además, utiliza 100 épocas y una configuración de red distinta de la implementación original empleada en este trabajo.

Los estudios generales de representaciones tampoco cierran este hueco, puesto que emplean aprendizaje por refuerzo o clonación de comportamiento sobre bancos de pruebas heterogéneos. Ninguno mantiene fija una *Diffusion Policy* para comparar estas alternativas bajo un protocolo común de Push-T.

Por tanto, en el corpus revisado no se ha identificado una comparación conjunta de cinco configuraciones concretas: ResNet-18 desde cero, ResNet-18 preentrenada en ImageNet congelada y ajustada, DINOv2 congelado y CLIP congelado. Tampoco se informa su relación entre rendimiento, parámetros entrenables, tiempo y memoria bajo un protocolo único de Push-T.

Este hueco justifica un estudio controlado que mantenga constante la política generativa, el conjunto de demostraciones y la evaluación. La variable experimental queda así restringida al codificador y a su estrategia de entrenamiento. El Capítulo 3 presenta el diseño empleado para realizar esa comparación.

3. METODOLOGÍA

Este capítulo describe el diseño experimental empleado para responder a los objetivos del Capítulo 1. En primer lugar, expone el planteamiento comparativo y las variables implicadas. A continuación, detalla la tarea, el conjunto de datos y la arquitectura de la política. Por último, define las cinco variantes evaluadas, los protocolos de entrenamiento y evaluación, las métricas, el entorno de ejecución y el coste temporal de los experimentos.

3.1 Planteamiento experimental

El estudio adopta un diseño comparativo centrado en una variable de interés. Esa variable es el bloque visual de la política, entendido como el codificador, sus pesos y su cadena de preprocesado. La arquitectura generativa, los datos y la evaluación permanecen constantes; el presupuesto y el tamaño de lote presentan las desviaciones descritas en el Apartado 3.5.

De este planteamiento se derivan tres hipótesis de trabajo. La primera sostiene que un codificador preentrenado sobre un corpus amplio de imágenes aporta representaciones más útiles que uno entrenado desde cero, puesto que el número de demostraciones disponibles es reducido. La segunda sostiene que el ajuste fino del codificador preentrenado mejora al codificador congelado, ya que adapta la representación a la distribución visual sintética de la tarea. La tercera sostiene que los codificadores basados en transformador de visión (*Vision Transformer*, ViT) alcanzan un rendimiento comparable al de las redes convolucionales, pero con un coste de inferencia superior.

Para que la comparación resulte interpretable, se mantienen constantes cinco factores: la red generativa y sus hiperparámetros, el conjunto de datos y su partición, la semilla de entrenamiento, el protocolo de evaluación y la dimensión del vector de condicionamiento a la salida del codificador. Esta última restricción resulta esencial: todas las variantes entregan un vector de 512 componentes, de modo que la red generativa recibe siempre una entrada de la misma dimensión.

Sin embargo, el diseño presenta una limitación asumida de forma explícita. Cada variante se entrena con una única semilla, por lo que no se cuantifica la variabilidad entre ejecuciones de entrenamiento. Esta decisión responde al coste computacional, ya que una ejecución ocupa entre 11 y 96 horas de la única unidad de procesamiento gráfico (*Graphics Processing Unit*, GPU) disponible, según se detalla en el Apartado 3.9.

3.2 Tarea y conjunto de datos

La tarea de evaluación es Push-T, propuesta junto con *Diffusion Policy* [3]. Un efector circular debe empujar una pieza rígida con forma de T sobre un plano hasta hacerla coincidir con una silueta objetivo. La tarea exige contacto intermitente y admite varias secuencias de empuje válidas, de ahí su carácter multimodal.

La política observa dos modalidades: una imagen en color de 96×96 píxeles renderizada desde una vista cenital y la posición bidimensional del efector. La acción es la posición

objetivo del efector, es decir, se emplea control en posición y no en velocidad. El entorno resuelve la dinámica de contacto mediante un motor de cuerpos rígidos bidimensional.

El conjunto de datos es el publicado por los autores del método, en formato `zarr` y con un tamaño aproximado de 30 megabytes. Contiene 206 demostraciones teleoperadas por un operador humano, que suman 25.650 transiciones. Cada transición almacena la imagen, el estado del efector, los puntos característicos de la pieza, la acción ejecutada y el número de contactos.

La partición de los datos reproduce la de la implementación de referencia. Un dos por ciento de los episodios se reserva para validación y el entrenamiento se limita a 90 episodios; los restantes se descartan. Mantener este criterio permite comparar los resultados con los publicados y, además, sitúa el experimento en el régimen de pocas demostraciones que motiva las hipótesis del Apartado 3.1.

Conviene precisar que la evaluación no consume datos del fichero anterior. El rendimiento se mide ejecutando la política en el simulador sobre condiciones iniciales generadas de forma independiente, según se detalla en el Apartado 3.6.

3.3 Arquitectura de la política

Diffusion Policy modela la distribución condicional $p(\mathbf{A}_t \mid \mathbf{O}_t)$, donde $\mathbf{O}_t$ agrupa las observaciones de los últimos T_o instantes y $\mathbf{A}_t$ es una secuencia de T_p acciones futuras [3]. El muestreo de esa distribución se realiza mediante un modelo probabilístico de difusión con eliminación de ruido (*Denoising Diffusion Probabilistic Model*, DDPM) [4].

El entrenamiento añade ruido gaussiano ϵ^k a una secuencia de acciones real $\mathbf{A}_t^0$ y ajusta una red ϵ_θ para predecirlo. La función de pérdida es el error cuadrático medio sobre el ruido, recogido en la Ecuación (3).

$$\mathcal{L} = \mathbb{E} \left[\|\epsilon^k - \epsilon_\theta(\mathbf{O}_t, \mathbf{A}_t^0 + \epsilon^k, k)\|^2 \right] \quad (3)$$

En la Ecuación (3), el índice k identifica el paso de difusión y determina la magnitud del ruido inyectado. La observación $\mathbf{O}_t$ actúa como condicionamiento, no como variable generada, lo que reduce el coste de inferencia. Por tanto, la calidad de la representación visual condiciona directamente lo que la red generativa puede aprender.

Durante la inferencia, la política parte de una secuencia de ruido puro y aplica K pasos de eliminación de ruido según la Ecuación (4).

$$\mathbf{A}_t^{k-1} = \alpha \left(\mathbf{A}_t^k - \gamma \epsilon_\theta(\mathbf{O}_t, \mathbf{A}_t^k, k) \right) + \mathcal{N}(0, \sigma^2 I) \quad (4)$$

Los coeficientes α , γ y σ de la Ecuación (4) proceden del planificador de ruido. El ruido fresco que se añade en cada paso preserva el carácter estocástico del muestreo y, con ello, la capacidad de representar varios modos de acción. En este trabajo se emplea el planificador DDPM con 100 pasos tanto en entrenamiento como en inferencia, sin acelerarlo mediante muestreadores implícitos.

Las acciones generadas se aplican bajo un esquema de horizonte deslizante. La política recibe $T_o = 2$ observaciones, predice $T_p = 16$ acciones y ejecuta únicamente las $T_a = 8$

primeras antes de replanificar [3]. Esta configuración equilibra la consistencia temporal de la trayectoria con la capacidad de reaccionar ante cambios en la escena.

La red ϵ_θ es una red convolucional unidimensional con arquitectura en U y dimensiones de canal 512, 1.024 y 2.048 en sus tres niveles de submuestreo. El vector de observación se inyecta como condicionamiento global en cada capa mediante modulación lineal por características (*Feature-wise Linear Modulation*, FiLM) [24]. Este bloque concentra unos 277 millones de parámetros y, por tanto, domina el coste de entrenamiento.

Delante de la red generativa opera el codificador de observaciones. Dicho módulo aplica las transformaciones de imagen propias de cada arquitectura, extrae un vector de características visuales y lo concatena con el estado del efector. La implementación de referencia utiliza una ResNet-18 [15] entrenada desde cero con dos modificaciones: sustituye el promediado global por un *spatial softmax*, que conserva la localización espacial de las activaciones [25], y reemplaza la normalización por lotes por normalización por grupos [26], ya que la primera interfiere con la media móvil exponencial de los pesos.

3.4 Variantes del codificador visual

Se definen cinco variantes, identificadas de V0 a V4, que modifican el bloque visual y el tratamiento de sus pesos. La Tabla 3 resume su configuración. Las tres primeras comparten arquitectura convolucional y permiten contrastar estrategias de entrenamiento; las dos últimas introducen transformadores de visión preentrenados con objetivos distintos.

Tabla 3. Configuración de las cinco variantes del codificador visual evaluadas.

Variante	Codificador visual	Pesos iniciales	Estrategia	Parámetros (millones)	
				Totales	Entrenables
V0	ResNet-18	Aleatorios	Desde cero	288	288
V1	ResNet-18	ImageNet-1k	Congelado	288	277
V2	ResNet-18	ImageNet-1k	Ajuste fino	288	288
V3	DINOv2 ViT-S/14	LVD-142M	Congelado	299	277
V4	CLIP ViT-B/16	WIT-400M	Congelado	363	277

Fuente: elaboración propia.

V0: línea base entrenada desde cero

La variante V0 reproduce la configuración original de *Diffusion Policy* [3]. Emplea una ResNet-18 con pesos aleatorios, opera sobre la imagen a su resolución nativa de 96 píxeles y aplica recorte aleatorio como único aumento de datos. El codificador se entrena de extremo a extremo junto con la red generativa y constituye la referencia frente a la que se comparan las demás variantes.

V1 y V2: transferencia desde ImageNet

Las variantes V1 y V2 parten de una ResNet-18 preentrenada sobre ImageNet-1k [16] y difieren en el tratamiento de sus pesos. En V1 el codificador permanece congelado, de modo que actúa como extractor fijo de características; en V2 sus pesos se actualizan junto con el

resto de la política. Ambas redimensionan la imagen a 224 píxeles y aplican la normalización estándar de ImageNet, condiciones bajo las que se obtuvieron los pesos originales.

El contraste entre V1 y V2 aísla el efecto de la estrategia de entrenamiento, puesto que ambas comparten arquitectura, pesos iniciales y cadena de preprocesado. El contraste entre V0 y las dos variantes preentrenadas no aísla, en cambio, el efecto de la inicialización, ya que el preprocesado viaja con los pesos. V0 opera a 96 píxeles, aplica recorte aleatorio y agrega las activaciones mediante *spatial softmax*, mientras que V1 y V2 operan a 224 píxeles, prescinden del recorte y entregan un descriptor global. Cuatro factores varían así de forma conjunta, de modo que esa comparación debe leerse sobre el bloque formado por el codificador y su preprocesado. El entrenamiento que los separaría se describe en el Apartado 5.2.

V3 y V4: transformadores de visión preentrenados

Las variantes V3 y V4 sustituyen la red convolucional por un transformador de visión [17]. V3 utiliza DINOv2 ViT-S/14, preentrenado de forma autosupervisada sobre el corpus LVD-142M [18]. V4 utiliza CLIP ViT-B/16, preentrenado con supervisión de lenguaje natural sobre pares de imagen y texto [19]. En ambos casos los pesos permanecen congelados, decisión que impone la memoria gráfica disponible. La medida recogida en la Tabla 11 la respalda: el ajuste fino de la ResNet-18 de V2, con once millones de parámetros en el codificador, ya reclama más memoria de la que tiene la tarjeta y solo se completa porque el controlador respalda el exceso con memoria del sistema, a costa de triplicar el tiempo por época. Los dos transformadores aportan 22 y 86 millones de parámetros a ese bloque, según se deduce de la Tabla 3, de modo que ajustarlos agravaría el problema en lugar de aliviarlo.

Dado que la dimensión de salida de estos modelos no coincide con la de la ResNet-18, se añade una capa lineal de proyección que reduce el descriptor a 512 componentes. Dicha capa sí se entrena, puesto que forma parte de la política y no del modelo preentrenado. Los codificadores preentrenados se cargan mediante la biblioteca `timm` [27], salvo la ResNet-18 de V2, que procede de `torchvision`.

3.5 Protocolo de entrenamiento

Las variantes comparten los hiperparámetros recogidos en la Tabla 4, salvo las dos excepciones indicadas en ella. El presupuesto se redujo durante la campaña conforme aumentaba el coste observado: 500 épocas para V0 y V1, 300 para V2 y V3, y 200 para V4. Estos valores quedan muy por debajo de las 3.000 épocas del artículo original. Dentro de cada presupuesto se aplica el criterio de parada que se precisa más adelante.

Además del presupuesto, el tamaño de lote de V3 y V4 se redujo a la mitad, ya que los dos transformadores operan a 224 píxeles y comprometen la memoria gráfica disponible con lotes de 64 muestras. La reducción se compensa acumulando el gradiente durante dos lotes antes de cada actualización, de modo que el tamaño de lote efectivo y el número de actualizaciones por época coinciden con los de V0, V1 y V2: 336 lotes y 168 actualizaciones frente a 168 lotes y otras tantas actualizaciones. Lo que sí cambia es el número de pasadas por el codificador, que se duplica, y con él el tiempo por época. La Tabla 11 matiza esta decisión: con lote reducido V4 llega al 90,3 % de la memoria de la tarjeta, de modo que la reducción estaba justificada, mientras que V3 se queda en el 78,2 % y disponía de más

Tabla 4. Hiperparámetros de entrenamiento de las cinco variantes.

Parámetro	Valor
Horizonte de predicción T_p	16 pasos
Pasos de observación T_o	2
Pasos de acción ejecutados T_a	8
Pasos de difusión (entrenamiento e inferencia)	100
Planificador de ruido	DDPM, varianza fija, predicción de ruido
Optimizador	AdamW [28]
Tasa de aprendizaje	1×10^{-4} , decaimiento coseno
Pasos de calentamiento	500
Media móvil exponencial de los pesos	Activada
Tamaño de lote	64 (32 en V3 y V4)
Acumulación de gradiente	1 (2 en V3 y V4)
Tamaño de lote efectivo	64 en las cinco variantes
Presupuesto máximo de épocas	500 (V0–V1); 300 (V2–V3); 200 (V4)
Semilla	42

Fuente: elaboración propia.

margen. En V3 la medida no sostiene la necesidad y la reducción debe leerse como cautela y como forma de igualar las condiciones de los dos transformadores.

Cada entrenamiento se lanza mediante un script de control que fija los parámetros de ejecución, verifica la memoria disponible e impide que se ejecuten dos procesos de forma simultánea. Esta salvaguarda responde a un incidente previo, en el que dos procesos concurrentes sobre el mismo directorio de salida agotaron la memoria del sistema y corrompieron el registro de métricas. El detalle de la configuración efectiva se recoge en el Capítulo A.

Durante el entrenamiento se guarda un punto de control cada 50 épocas y se conservan los tres mejores según la puntuación de evaluación. El modelo evaluado en el Capítulo 4 es el de mayor puntuación, criterio coherente con el empleado en la literatura de la tarea [8].

El criterio de parada anticipada se apoya en esos mismos puntos de control. Un entrenamiento se interrumpe cuando la puntuación de evaluación no supera su máximo en dos evaluaciones consecutivas, esto es, tras 100 épocas sin mejora, mientras la pérdida de entrenamiento sigue descendiendo. Esa combinación apunta a sobreajuste, de modo que prolongar la ejecución no aportaría un punto de control mejor. La parada no se aplica antes de la época 200, para no interrumpir una variante de convergencia lenta. El criterio no altera el modelo seleccionado, puesto que este se elige entre los puntos de control ya guardados.

Este criterio no precedió al experimento, sino que se adoptó durante su ejecución. Las dos primeras variantes, V0 y V1, agotaron 500 épocas porque entonces no existía regla de parada. La formulación anterior se fijó al observar sus curvas y se aplicó a V2, detenida tras 266 épocas completas. V3 se interrumpió manualmente en la época de índice 154, antes del mínimo de 200, mientras que V4 agotó sus 200 épocas. Las decisiones se tomaron por inspección, no de forma automática.

Esta asimetría afecta al coste y a la selección del modelo. V3 y V4 solo disponen de cuatro evaluaciones, frente a las diez de V0 y V1, por lo que tuvieron menos oportunidades de alcanzar un máximo tardío. El Apartado 3.9 reconstruye el contrafactual bajo el criterio aplicado de forma uniforme, dentro del presupuesto asignado a cada variante.

3.6 Protocolo de evaluación

La evaluación se ejecuta cada 50 épocas sobre 56 condiciones iniciales del simulador. Seis de ellas proceden de la distribución de entrenamiento y las 50 restantes se generan con semillas de entorno reservadas, no empleadas durante el ajuste. Las primeras permiten estimar el ajuste a los datos vistos; las segundas constituyen la medida de rendimiento que se reporta.

Cada condición inicial se resuelve en un episodio completo de bucle cerrado. La política recibe las observaciones, genera una secuencia de acciones, ejecuta las primeras T_a y repite el ciclo hasta alcanzar el límite de pasos del episodio o completar la tarea. Los pesos utilizados son los de la media móvil exponencial, no los del modelo instantáneo, tal como prescribe la implementación de referencia.

Las condiciones se agrupan en tandas de ocho procesos para no agotar la memoria del sistema. Esta agrupación modifica el tiempo de ejecución de la evaluación, pero no las métricas resultantes, puesto que la agregación recorre las 56 condiciones con independencia del número de procesos simultáneos.

3.7 Métricas

La métrica principal es la puntuación de cobertura de la tarea Push-T [3]. Para un episodio j , se calcula en cada instante t la cobertura $c_{j,t}$, definida como la fracción del área objetivo cubierta por la pieza. La puntuación del episodio se obtiene según la Ecuación (5).

$$s_j = \max_{1 \leq t \leq T} \min\left(\frac{c_{j,t}}{\tau}, 1\right) \quad (5)$$

En la Ecuación (5), T es la duración máxima del episodio y τ el umbral de cobertura que define el éxito, fijado en 0,95 por la implementación de referencia. La puntuación resulta así continua y acotada en el intervalo unidad: vale uno cuando la pieza alcanza el umbral en algún instante y crece de forma proporcional a la cobertura en caso contrario. Se toma el máximo a lo largo del episodio, y no el valor final, porque la pieza puede desplazarse tras alcanzar la posición objetivo.

La métrica que se reporta es la media de s_j sobre los 50 episodios evaluados, denominada puntuación media de evaluación. Junto a ella se reportan la desviación típica y el error estándar de esos 50 valores, magnitudes que describen la dispersión de la propia métrica. Se registran además cuatro cantidades auxiliares: la pérdida de entrenamiento, la pérdida de validación, el error cuadrático medio de la acción predicha y la puntuación media sobre las condiciones de entrenamiento. Las dos primeras describen el ajuste, mientras que la comparación entre las dos puntuaciones permite detectar sobreajuste.

El nombre de la métrica principal requiere una precisión. Las 50 condiciones se generan con semillas reservadas y no intervienen en el ajuste de los pesos, pero se evalúan cada 50 épocas y determinan qué punto de control se conserva. Cumplen, por tanto, la función de un conjunto de selección y no la de una evaluación final independiente. El término *prueba* se reserva en esta memoria para una medida sobre condiciones ajenas a la selección, que este trabajo no realiza; el sesgo que se deriva de ello se analiza en el Apartado 4.7.

La dispersión entre episodios permite acompañar cada diferencia observada de un intervalo de confianza y contrastarla mediante la prueba de los rangos con signo de Wilcoxon, aplicada sobre los pares de episodios comunes a dos variantes. Ambas medidas caracterizan la variabilidad debida a las condiciones iniciales del simulador. No sustituyen, en cambio, a la variabilidad entre semillas de entrenamiento, que exigiría repetir los ajustes y queda fuera del alcance fijado en el Apartado 1.4.

Al comparar las cinco variantes se realizan diez contrastes por pares. Sus valores p se ajustan mediante el procedimiento secuencial de Holm, que controla el error familiar sin suponer independencia entre contrastes. La significación se determina sobre los valores ajustados con un umbral de 0,05.

El coste computacional se caracteriza mediante cuatro indicadores. El primero es el número de parámetros totales y entrenables de cada variante, recogido en la Tabla 3. El segundo es el tiempo por paso de optimización, medido tras la fase de calentamiento y agregado por época en la Tabla 7. El tercero es la latencia de inferencia para un lote de tamaño fijo. El cuarto es el pico de memoria gráfica reservada durante el entrenamiento. Los dos últimos se reportan en el Apartado 4.4.

La latencia se desglosa en tres magnitudes, porque una sola oculta el efecto que se estudia. Cada llamada a la política ejecuta el codificador visual una vez y la red generativa cien veces, de modo que el tiempo total lo domina un componente idéntico en las cinco variantes. Se reportan, por tanto, la latencia de la llamada completa, la del codificador aislado y la de la llamada dividida entre las ocho acciones que emite, esta última porque acota la frecuencia de control alcanzable. Las medidas se toman en precisión `float32`, con lotes de 1 y 8 muestras, y se resumen mediante la mediana y el recorrido intercuartílico de 50 repeticiones cronometradas, precedidas de 20 descartadas. Las repeticiones se alternan entre variantes por rondas, para que la deriva térmica del equipo portátil afecte por igual a todas.

El pico de memoria exige reproducir el paso de optimización completo, ya que los gradientes, los momentos de AdamW y la copia de la media móvil son justamente lo que ocupa la tarjeta. Se ejecutan doce pasos sobre lotes reales, con el tamaño propio de cada variante, y se registra la memoria reservada por el asignador. Se acompaña del pico en inferencia, que describe lo que exigiría un despliegue.

Ambas medidas se toman en entornos de ejecución distintos y esa diferencia se declara en cada tabla. La latencia se mide en el entorno de inferencia sobre Windows y el pico de memoria en el entorno de WSL en el que se entrenó, puesto que el asignador de memoria de PyTorch cambió entre ambas versiones y una cifra tomada en Windows no describiría las ejecuciones recogidas en la Tabla 7. Dentro de cada indicador, las cinco variantes comparten entorno.

3.8 Entorno de ejecución y reproducibilidad

Todos los experimentos se ejecutan sobre un único equipo portátil, cuyas características se recogen en la Tabla 5. El cómputo no se reparte entre varias máquinas ni se recurre a servicios en la nube, de modo que las medidas de coste del Apartado 3.9 son directamente comparables entre variantes.

La restricción determinante es la memoria gráfica de ocho gigabytes. Dicho límite condiciona el tamaño de lote de V4 y descarta el ajuste fino de los transformadores de visión, según se

Tabla 5. Plataforma de ejecución de los experimentos.

Bloque	Componente	Características
Cómputo	Procesador	Intel Core i9-12900H, 16 núcleos lógicos
	GPU	NVIDIA GeForce RTX 3070 Ti Laptop, 8 GB
	Capacidad CUDA	8.6 (Ampere), controlador 596.21
Memoria	RAM física	23,7 GB
	RAM del subsistema	18 GB, más 16 GB de intercambio
Plataforma	Sistema anfitrión	Windows 11
	Subsistema	WSL 2.5.9.0, núcleo 6.6.87.2
	Distribución	Ubuntu 24.04.2 LTS
Almacenamiento	Disco de trabajo	1.007 GB virtuales (ext4 sobre VHDX)

Fuente: elaboración propia.

justificó en el Apartado 3.4. La memoria principal impone una segunda restricción menos evidente: el conjunto de datos se carga íntegramente en RAM y ocupa unos 2,8 GB por proceso, cifra que obliga a limitar los procesos de carga y de simulación. La asignación de memoria al subsistema se amplió de 13 a 18 GB antes de ejecutar V2, ya que el ajuste fino del codificador aumenta el consumo respecto a las variantes congeladas.

Sobre esa plataforma se despliega la pila de software detallada en la Tabla 6. Las versiones están fijadas y no se actualizan entre variantes, puesto que la comparación exige que el único factor que cambie sea el codificador visual.

Tabla 6. Componentes de la pila de software y función de cada uno.

Componente	Versión	Función en el experimento
Python	3.9.15	Intérprete, en entorno <code>conda</code> aislado
PyTorch	1.12.1	Marco de trabajo de aprendizaje profundo [29]
CUDA	11.6	Biblioteca de cálculo sobre GPU
torchvision	0.13.1	ResNet-18 y pesos de ImageNet-1k
timm	0.9.16	DINOv2 y CLIP preentrenados [27]
diffusers	0.11.1	Planificador de ruido DDPM
huggingface_hub	0.25.2	Descarga de los pesos preentrenados
robomimic	0.2.0	Utilidades de clonación de comportamiento [8]
diffusion_policy	Bifurcación propia	Política, entorno Push-T y evaluación
Hydra	1.2.0	Configuración y registro de cada ejecución
Zarr	2.12.0	Formato del conjunto de demostraciones

Fuente: elaboración propia.

El código de partida es la implementación pública de *Diffusion Policy* [3], sobre la que se añaden los codificadores de V1 a V4 y sus configuraciones. Las modificaciones se concentran en el módulo del codificador de observaciones, mientras que la red generativa, el bucle de entrenamiento y el evaluador permanecen sin cambios. Por tanto, cualquier diferencia entre variantes es atribuible al codificador y no a la infraestructura que lo rodea.

La reproducibilidad se apoya en tres medidas. En primer lugar, la semilla global se fija en 42 para todas las variantes y las condiciones de evaluación emplean semillas reservadas y constantes. En segundo lugar, las versiones exactas de las bibliotecas se documentan en el Capítulo A. En tercer lugar, la configuración efectiva de cada ejecución se almacena junto a

sus resultados.

No obstante, la reproducibilidad no es exacta. Los algoritmos deterministas de la biblioteca de aceleración no están activados, ya que resultan incompatibles con las operaciones convolucionales empleadas. En consecuencia, dos ejecuciones con la misma semilla pueden diferir ligeramente, y las diferencias pequeñas entre variantes deben interpretarse con cautela. El Capítulo 4 aplica ese criterio al comparar las puntuaciones obtenidas.

3.9 Coste temporal de los entrenamientos

El coste de cómputo condiciona el alcance del estudio y merece, por ello, una cuantificación explícita. La Tabla 7 recoge la duración de cada ejecución sobre la plataforma descrita en el Apartado 3.8. Se distinguen dos magnitudes: el tiempo del bucle de optimización, medido sobre las épocas de entrenamiento, y el tiempo total transcurrido, que añade la validación por época, las evaluaciones en el simulador y el guardado de puntos de control.

Tabla 7. Coste temporal de los entrenamientos ejecutados.

Variante	Épocas registradas	Bucle por época	Bucle acumulado	Tiempo total
V0	500	1,6 min	13,0 h	17,4 h
V1	500	5,3 min	43,8 h	96,5 h
V2	266	14,7 min	65,3 h [†]	84,9 h [†]
V3	155	2,3 min	5,9 h	10,9 h
V4	200	4,0 min	13,3 h	27,4 h
Total	1.621		141,3 h	237,1 h

[†] Valor estimado a partir del tiempo medio por época de las épocas efectivamente cronometradas.

Fuente: elaboración propia.

Las cinco variantes suman 237,1 h de cómputo, muy por encima de las 75 horas que cabría proyectar a partir de la primera ejecución. La discrepancia procede de dos factores. En primer lugar, el coste por época crece casi un orden de magnitud entre V0 y V2. En segundo lugar, la validación y las evaluaciones en el simulador amplían el tiempo del bucle de optimización, puesto que los episodios se ejecutan en el procesador.

Las dos últimas columnas no admiten, sin embargo, una lectura directa como medida de coste relativo. V2 acumula menos tiempo total que V1 pese a ser casi tres veces más cara por época, simplemente porque se detuvo antes. La comparación entre variantes debe apoyarse, por tanto, en el tiempo por época, mientras que el tiempo total mide lo que ha consumido el experimento tal como se ejecutó.

Tres aclaraciones acotan la lectura de la tabla. En primer lugar, solo V2 conserva el cronómetro de una parte de sus épocas, de modo que su tiempo acumulado se extrapola a partir de la media; el símbolo [†] marca los valores estimados. En segundo lugar, V2 se ejecutó en dos sesiones separadas por una interrupción de dos días, que se descuenta del tiempo total. En tercer lugar, V3 se detuvo tras registrar 155 épocas de un presupuesto de 300, mientras que V4 completó las 200 asignadas.

Este coste explica dos decisiones del diseño experimental. La primera es el uso de una única semilla por variante, ya que replicar las cinco condiciones tres veces exigiría del orden de 700 horas de la misma GPU. La segunda es la adopción del criterio de parada anticipada del

Apartado 3.5: V1 dedicó unas 70 horas a épocas posteriores a su mejor punto de control, sin obtener ninguna mejora.

La tabla mide el experimento tal como se ejecutó y no un protocolo fijado de antemano. Bajo el criterio uniforme, V0 se habría detenido en la época 300 y V1 en la 250. Las cinco ejecuciones habrían sumado 111,4 h de bucle y 177,3 h totales, frente a 141,3 h y 237,1 h. El punto de control seleccionado solo cambiaría en V0, que conservaría el de la época 200 con 0,8643 en lugar del de la 350 con 0,8645. Ninguna conclusión del Capítulo 4 depende de esa diferencia.

4. RESULTADOS

Este capítulo presenta los resultados de las cinco variantes y los relaciona con las hipótesis formuladas en el Apartado 3.1. El análisis comienza con la cobertura de las ejecuciones, continúa con la evolución del rendimiento y de las pérdidas, y concluye con una comparación cualitativa y las limitaciones de la evidencia.

4.1 Cobertura de las ejecuciones

La Tabla 8 resume la cobertura de los registros y el mejor valor de la métrica principal. V0 y V1 agotaron 500 épocas, V2 completó 266, V3 registró 155 y V4 agotó su presupuesto de 200. Los dos transformadores solo disponen de cuatro evaluaciones, frente a las diez de V0 y V1.

Tabla 8. Cobertura de los entrenamientos y mejor puntuación media de evaluación.

Variante	Estrategia	Épocas	Eval.	Mejor puntuación	IC 95 %	Época
V0	Desde cero	500	10	$0,864 \pm 0,040$	[0,787; 0,942]	350
V1	Congelado	500	10	$0,668 \pm 0,053$	[0,564; 0,771]	150
V2	Ajuste fino	266	6	$0,648 \pm 0,057$	[0,536; 0,759]	150
V3	Congelado	155	4	$0,622 \pm 0,053$	[0,518; 0,727]	100
V4	Congelado	200	4	$0,535 \pm 0,059$	[0,419; 0,651]	100

La puntuación se acompaña de su error estándar sobre los 50 episodios evaluados. En V2 se excluye la época de índice 266, interrumpida tras cinco lotes y sin validación. V3 se detuvo en el índice 154.

Fuente: elaboración propia.

El valor reportado es el máximo de una serie de evaluaciones y, por tanto, un estimador optimista. La media de las tres últimas ofrece una lectura menos sesgada: 0,862 en V0, 0,547 en V1, 0,585 en V2, 0,572 en V3 y 0,483 en V4. V0 permanece estable, mientras que las cuatro variantes preentrenadas retroceden después de su máximo. El sesgo del máximo puede favorecer a V0 y V1 porque acumulan más evaluaciones; la media final mantiene, no obstante, la ventaja de V0.

V2 se reanudó desde un punto de control intermedio. El registro conserva tanto cinco épocas de la rama abandonada como sus equivalentes tras la reanudación, con pasos globales repetidos. Para construir las curvas se mantuvo la primera sesión hasta la época 69 y la rama reanudada desde la 70. Este criterio evita promediar estados del optimizador que no pertenecen a una misma trayectoria de entrenamiento.

4.2 Evolución del rendimiento

La Figura 1 representa exclusivamente las evaluaciones oficiales sobre las 50 condiciones reservadas. Las barras verticales indican el error estándar de la media sobre esos episodios. Los marcadores se sitúan cada 50 épocas y las estrellas identifican el punto de control seleccionado para cada variante.

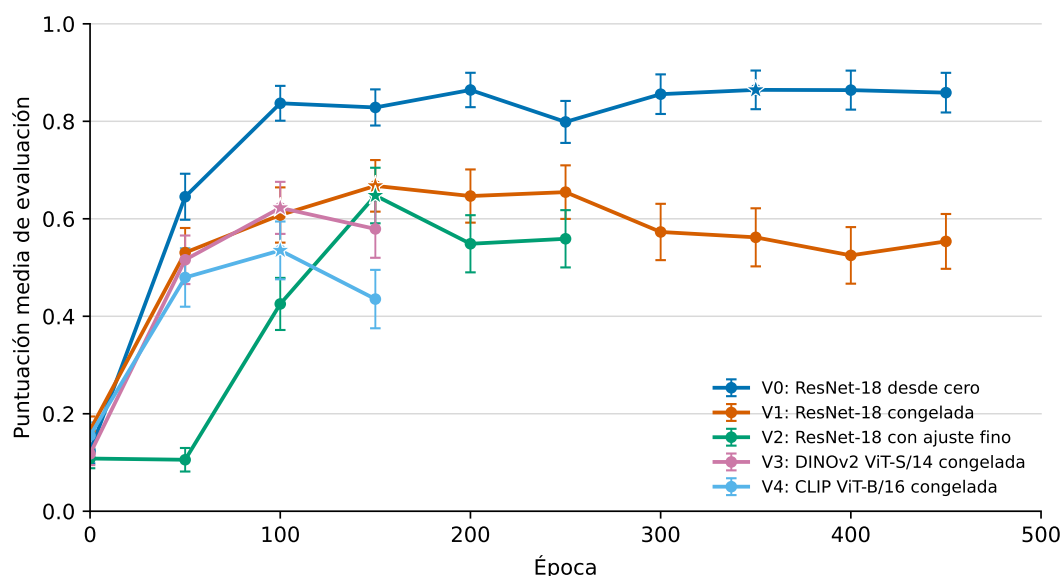


Figura 1. Evolución de la puntuación media de evaluación de las cinco variantes.

Fuente: elaboración propia.

V0 alcanza 0,8370 en la época 100 y permanece cerca de 0,86 durante la mayor parte del entrenamiento posterior. Su máximo, 0,8645, aparece en la época 350; la última evaluación, en la 450, conserva 0,8588. La pequeña diferencia entre ambos valores indica que la selección del punto de control no depende de un pico aislado.

V1 mejora hasta 0,6676 en la época 150. Después oscila brevemente alrededor de 0,65 y desciende hasta 0,5536 en la última evaluación. Esta reducción equivale a 0,1140 puntos, un 17,1 % respecto a su máximo. En consecuencia, las 350 épocas restantes no producen un punto de control mejor.

V2 presenta una convergencia más lenta. Su puntuación permanece alrededor de 0,11 hasta la época 50, asciende a 0,4253 en la 100 y alcanza 0,6477 en la 150. Las dos evaluaciones posteriores caen a 0,5487 y 0,5590. Este comportamiento motiva la detención anticipada, puesto que prolongar la ejecución no había recuperado el máximo tras 100 épocas.

V3 progresa de 0,1177 a 0,6224 entre las épocas 0 y 100. La evaluación de la época 150 desciende a 0,5792, y la ejecución se detiene cuatro épocas después. V4 alcanza 0,5351 en la época 100 y cae a 0,4353 en la 150, antes de completar las 200 épocas asignadas.

La comparación de los mejores puntos de control sitúa V0 por encima de las demás variantes. Los márgenes abarcan desde 0,197 frente a V1 hasta 0,329 frente a V4. La Tabla 9 relaciona las diez diferencias con su dispersión entre episodios y con el contraste de Wilcoxon.

Las cuatro diferencias entre V0 y las variantes preentrenadas excluyen el cero y conservan significación tras el ajuste de Holm. Entre las variantes preentrenadas, solo V1 frente a V4 alcanza significación sin corregir, con $p = 0,0084$. El ajuste eleva ese valor a 0,0501, por lo que ningún contraste interno supera el umbral familiar del 5 %. Estos resultados no cuantifican la variabilidad entre semillas de entrenamiento, cuestión que se retoma en el Apartado 4.7.

Tabla 9. Diferencias pareadas entre los puntos de control seleccionados.

Comparación	Diferencia media	IC 95 %	p	p_{Holm}
V0 frente a V1	0,197	[0,091; 0,303]	0,0004	0,0030
V0 frente a V2	0,217	[0,102; 0,332]	0,0001	0,0009
V0 frente a V3	0,242	[0,122; 0,362]	0,0003	0,0026
V0 frente a V4	0,329	[0,208; 0,451]	0,000003	0,000030
V1 frente a V2	0,020	[-0,093; 0,133]	0,82	1,00
V1 frente a V3	0,045	[-0,089; 0,179]	0,46	1,00
V1 frente a V4	0,132	[0,018; 0,247]	0,0084	0,0501
V2 frente a V3	0,025	[-0,101; 0,152]	0,54	1,00
V2 frente a V4	0,113	[-0,034; 0,259]	0,10	0,5154
V3 frente a V4	0,087	[-0,027; 0,201]	0,14	0,5532

Diferencias calculadas sobre los 50 episodios comunes. La prueba de Wilcoxon emplea la aproximación normal y los diez valores p se ajustan mediante el procedimiento de Holm. Los intervalos no incorporan este ajuste.

Fuente: elaboración propia.

4.3 Ajuste y generalización

La puntuación de la tarea debe interpretarse junto con las pérdidas del modelo. La Figura 2 muestra los valores agregados al final de cada época completa. La escala logarítmica permite observar simultáneamente la fase inicial y los cambios de menor magnitud al final del entrenamiento.

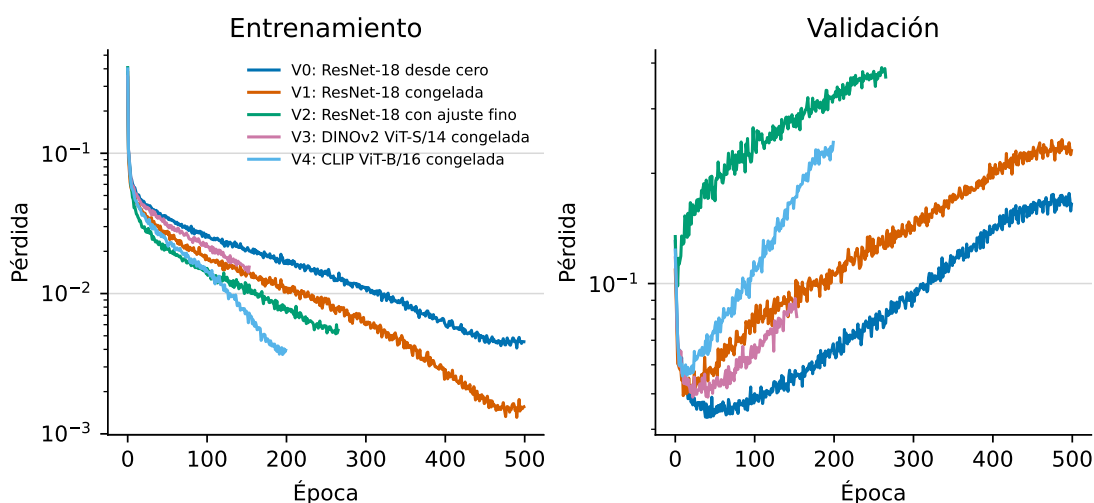


Figura 2. Evolución de las pérdidas de entrenamiento y validación.

Fuente: elaboración propia.

Las cinco variantes reducen de forma sostenida la pérdida de entrenamiento. Sin embargo, la pérdida de validación vuelve a crecer después de las primeras épocas. El patrón es especialmente claro en V1: entre las épocas 150 y 450, la pérdida de entrenamiento baja de 0,0145 a 0,00166, mientras que la de validación sube de 0,0869 a 0,2247 y la puntuación de evaluación disminuye.

V2 muestra la misma divergencia. Entre las épocas 150 y 250, su pérdida de entrenamiento pasa de 0,0115 a 0,00587, la de validación aumenta de 0,277 a 0,380 y la puntuación pierde 0,089 puntos. Ambos casos son compatibles con sobreajuste. V0 también incrementa su pérdida de validación, pero su puntuación permanece estable; por tanto, la pérdida de

predicción del ruido no actúa como sustituto directo del rendimiento en bucle cerrado.

V3 y V4 reproducen esa divergencia. Entre las épocas 100 y 150, la pérdida de entrenamiento de V3 baja de 0,0218 a 0,0155, mientras la de validación sube de 0,0681 a 0,0865. En V4, las mismas magnitudes pasan de 0,0143 a 0,00741 y de 0,110 a 0,171. Ambas puntuaciones disminuyen a la vez.

El error cuadrático medio de la acción refuerza esa distinción. En los puntos de control seleccionados vale 116,17 para V0, 147,31 para V1, 42,86 para V2, 314,78 para V3 y 94,93 para V4. V2 obtiene el menor error, pero no la mejor puntuación. La precisión sobre acciones registradas y el control en bucle cerrado miden, por tanto, propiedades diferentes.

4.4 Coste computacional

Los tiempos recogidos en la Tabla 7 muestran diferencias amplias. El bucle de V0 requiere 1,6 min por época, frente a 5,3 min en V1, 14,7 min en V2, 2,3 min en V3 y 4,0 min en V4. Estos valores multiplican el coste de la línea base por factores entre 1,5 y 9,5.

V2 añade la retropropagación a través del codificador y presenta el mayor coste por época. V3 y V4 duplican el número de lotes al reducir su tamaño a 32 muestras, si bien la acumulación de gradiente mantiene las actualizaciones por época en las 168 comunes a las cinco variantes. V3 muestra, sin embargo, que operar a 224 píxeles no determina por sí solo el tiempo: su codificador congelado queda más cerca de V0 que de V1.

Conviene no confundir ese coste unitario con el tiempo total. V2 acumula 84,9 h frente a las 96,5 h de V1 pese a ser la variante más cara por época, porque se detuvo antes. V3 solo suma 10,9 h por la misma razón. La comparación válida entre variantes es el tiempo por época; el total de 237,1 h describe el experimento ejecutado con presupuestos distintos.

Latencia de inferencia

El tiempo de entrenamiento no describe el coste de usar la política una vez ajustada. La Tabla 10 recoge la latencia de una llamada completa, medida sobre los puntos de control seleccionados y bajo el protocolo del Apartado 3.7.

Tabla 10. Latencia de inferencia de las cinco variantes, en milisegundos.

Variante	Píxeles	Lote de 1 muestra			Lote de 8 muestras		
		Llamada	Codificador	Por acción	Llamada	Codificador	Por acción
V0	76	1.743,9	3,0	218,0	2.080,8	3,1	260,1
V1	224	1.727,1	2,7	215,9	2.094,6	10,5	261,8
V2	224	1.723,8	2,8	215,5	2.096,2	10,6	262,0
V3	224	1.731,9	8,4	216,5	2.168,3	44,1	271,0
V4	224	1.749,1	16,1	218,6	2.253,9	131,8	281,7

Mediana de 50 repeticiones cronometradas, precedidas de 20 descartadas y alternadas entre variantes por rondas. El recorrido intercuartílico no supera los 82,6 ms en la columna de la llamada ni los 11,6 ms en la del codificador. Condiciones: 100 pasos de difusión, 2 observaciones de entrada, 8 acciones de salida, precisión float32, entorno de inferencia sobre Windows con PyTorch 2.6.0 y CUDA 12.4. La columna de píxeles indica la resolución que recibe el codificador.

Fuente: elaboración propia.

La llamada completa apenas distingue las variantes. Con lote unitario oscila entre 1.723,8 ms en V2 y 1.749,1 ms en V4, un margen del 1,5% que queda dentro de la dispersión de las propias medidas. La causa es estructural: cada llamada ejecuta el codificador una vez y la red generativa cien, de modo que el tiempo lo domina un componente idéntico en las cinco variantes. Una tabla que solo recogiera esa columna sugeriría que el codificador no influye en el coste de inferencia, conclusión que las dos columnas siguientes desmienten.

El codificador aislado sí ordena las variantes. Con lote unitario va de 2,7 ms en V1 a 16,1 ms en V4, un factor de 5,9. Con lote de ocho muestras la separación se amplía hasta un factor de 43, entre 3,1 ms en V0 y 131,8 ms en V4. El contraste entre ambos lotes también revela el efecto de la resolución: con una sola muestra, V0, V1 y V2 se separan menos de un milisegundo pese a operar a 76 y 224 píxeles, porque a ese tamaño domina el coste de lanzar los núcleos y no el de la aritmética; con ocho muestras, V0 mantiene sus 3,1 ms mientras V1 y V2 se van a 10,5 ms. En conjunto, el sobre coste del codificador existe y sigue el orden esperado, pero se diluye en el muestreo iterativo.

La tercera columna traduce el resultado a términos de control. Cada llamada emite ocho acciones, de modo que el coste por acción se sitúa entre 215,5 ms y 218,6 ms, equivalente a unas 4,6 acciones por segundo. El simulador consume esas ocho acciones a 10 Hz, esto es, en 0,8 s, frente a los 1,7 s que cuesta generarlas. Ninguna de las cinco variantes alcanza el control en tiempo real sobre este equipo con 100 pasos de difusión, y la diferencia entre codificadores no cambia esa conclusión: el margen está en el número de pasos del muestreo, sobre el que actúan los planificadores acelerados y la destilación revisados en el Apartado 2.2.

Memoria gráfica

El último indicador anunciado en el Apartado 3.7 es el pico de memoria gráfica reservada. La Tabla 11 lo recoge junto al pico en inferencia, que describe lo que exigiría un despliegue.

Tabla 11. Pico de memoria gráfica reservada, en gibibytes.

Variante	Codificador	Entrenamiento			Inferencia	
		Lote	Reservado	Ocupación	Lote 1	Lote 8
V0	Desde cero	64	6,447	80,6 %	1,348	1,307
V1	Congelado	64	6,896	86,2 %	1,355	1,395
V2	Ajuste fino	64	10,293	128,7 %	1,355	1,375
V3	Congelado	32	6,252	78,2 %	1,385	1,461
V4	Congelado	32	7,223	90,3 %	1,646	1,807

Memoria reservada por el asignador de PyTorch sobre una tarjeta de 8 gibibytes; la ocupación es su cociente con esa capacidad. Medida sobre 12 pasos de optimización completos, con el lote propio de cada variante, en el entorno de WSL empleado para entrenar, con PyTorch 1.12.1 y CUDA 11.6. Cada combinación de variante y modo se midió en un proceso independiente. V3 y V4 acumulan el gradiente durante dos lotes, de modo que su lote efectivo también es de 64 muestras.

Fuente: elaboración propia.

V2 es el caso extremo y el más informativo. Su paso de optimización reserva 10,293 gibibytes frente a los 8 de la tarjeta, un 128,7% de la capacidad disponible. La ejecución no aborta porque el controlador respalda el exceso con memoria del sistema, pero cada acceso que cae fuera de la tarjeta se paga en tiempo. Ahí está la explicación de sus 14,7 min por época,

casi el triple que V1, con la que comparte arquitectura, pesos iniciales, resolución y tamaño de lote. Lo único que las separa es la retropropagación a través del codificador.

Congelar el codificador, en cambio, no reduce el pico por sí solo. V1 reserva 6,896 gibibytes, por encima de los 6,447 de V0, porque opera a 224 píxeles en lugar de 76. Lo que la congelación ahorra son el gradiente y el estado del optimizador del codificador, once millones de parámetros frente a los 277 millones del resto de la política; las activaciones del paso hacia delante se calculan igual. La memoria la determinan, por tanto, la resolución de entrada y el tamaño de lote antes que la estrategia de entrenamiento, con la salvedad del ajuste fino, que añade el grafo de retropropagación completo.

Los dos transformadores se midieron con su lote de 32 muestras. V3 reserva 6,252 gibibytes y V4, 7,223, el 90,3 % de la tarjeta. Esa cifra de V4 respalda la reducción de lote descrita en el Apartado 3.5; la de V3 deja bastante más margen, de modo que en su caso la reducción se explica mejor como cautela y como forma de mantener idénticas las condiciones de los dos transformadores. En inferencia el panorama se invierte: las cinco variantes ocupan entre 1,307 y 1,807 gibibytes, esto es, entre el 16 y el 23 % de la tarjeta. El coste de memoria de este experimento es un problema de entrenamiento y no de despliegue.

4.5 Comprobación cualitativa

Los cuadernos de inferencia permiten inspeccionar trayectorias individuales de los mejores puntos de control. La Figura 3 compara V1 y V2 desde la misma condición inicial, generada con la semilla 10.049. Cada panel muestra el estado inicial, el estado final y la puntuación instantánea durante el episodio.

V1 alcanza una puntuación máxima de 0,9544, frente a 0,8818 en V2. Ninguna de las dos trayectorias llega al umbral unitario de éxito, aunque V1 consigue una superposición mayor. V0 sí alcanza 1,0 en su cuaderno, pero parte de la semilla 10.000; por ese motivo, su trayectoria no se incorpora a la comparación visual.

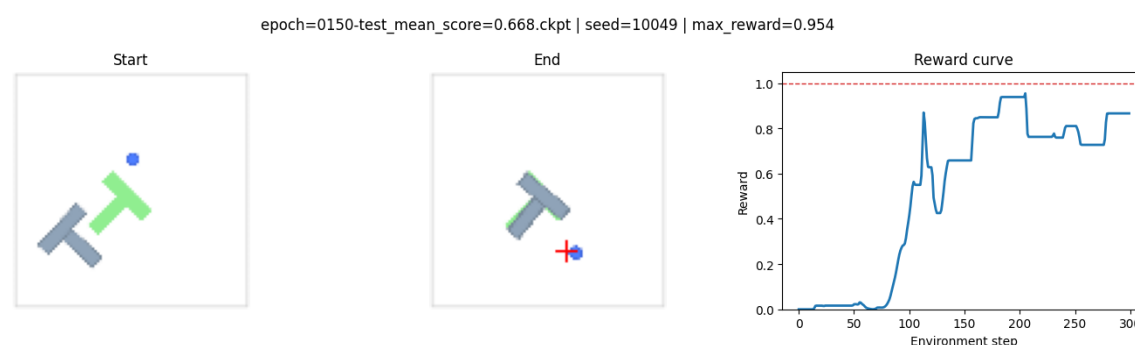
Estas demostraciones no sustituyen la evaluación cuantitativa. Sus semillas quedan fuera del intervalo oficial 100.000–100.049 y cada imagen representa un solo episodio. Además, la puntuación mostrada es el máximo temporal, no el valor del estado final. Su función es ilustrar el comportamiento que origina la métrica, no estimar el rendimiento medio.

4.6 Contraste de las hipótesis

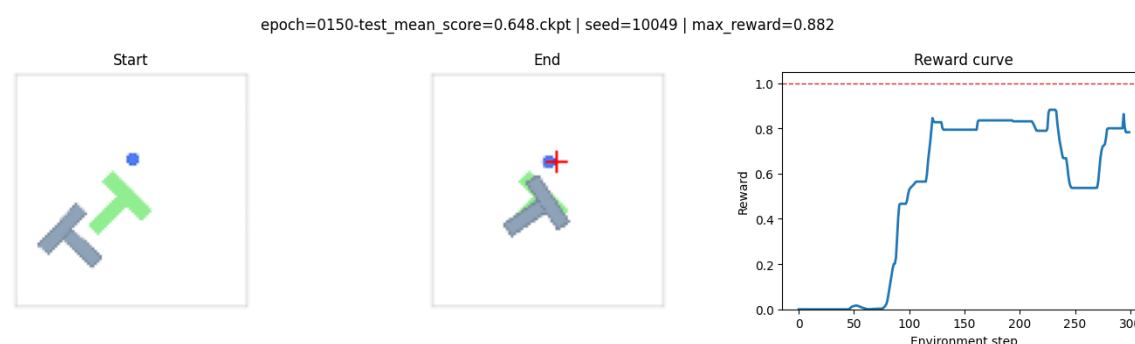
La primera hipótesis planteaba que el preentrenamiento aportaría representaciones más útiles que el entrenamiento desde cero. Los resultados no la respaldan: V0 supera entre 0,197 y 0,329 puntos a las cuatro variantes preentrenadas. Todas las diferencias conservan significación tras el ajuste de Holm.

La segunda hipótesis proponía una ventaja del ajuste fino sobre la congelación. Tampoco queda respaldada, ya que V1 obtiene 0,668 y V2 0,648. La diferencia de 0,020 no alcanza significación ($p = 0,82$) y su intervalo de confianza, $[-0,093; 0,133]$, contiene el cero. En consecuencia, tampoco cabe afirmar la superioridad de la congelación: lo que descarta la evidencia es una mejora observable del ajuste fino en esta ejecución.

La tercera hipótesis proponía que los transformadores alcanzarían un rendimiento comparable al de las redes convolucionales con mayor coste de inferencia. V3 y V4 quedan por



(a) V1, ResNet-18 preentrenada y congelada.



(b) V2, ResNet-18 preentrenada con ajuste fino.

Figura 3. Inferencia de V1 y V2 sobre una condición inicial compartida.

Fuente: elaboración propia.

debajo de V0, pero no se distinguen de V1 y V2 tras la corrección familiar. Por tanto, los datos no muestran una ventaja de rendimiento propia de la arquitectura ViT.

Su segunda parte, la relativa al coste, se cumple en el codificador y no en la política. La Tabla 10 sitúa el codificador de V3 y V4 entre tres y seis veces por encima del de las variantes convolucionales, y la distancia crece con el tamaño de lote. La llamada completa, en cambio, apenas varía un 1,5 % entre las cinco, porque los 100 pasos de difusión diluyen esa diferencia. El mayor coste de inferencia de los transformadores es real y medible, pero no se traduce en un coste apreciable de la política en la configuración evaluada.

4.7 Limitaciones

La primera limitación es la desigualdad del presupuesto de épocas. V0 y V1 acumulan diez evaluaciones, V2 seis, y V3 y V4 solo cuatro. V3 se interrumpió tras 155 épocas de las 300 previstas, de modo que no puede descartarse una recuperación posterior. La comparación corresponde a los mejores puntos observados dentro de esos presupuestos, no a curvas con igual horizonte.

La segunda limitación es el uso de una sola semilla de entrenamiento. Los intervalos y los contrastes de la Tabla 9 cuantifican la dispersión debida a las condiciones iniciales del simulador, ya que los 50 episodios son condiciones distintas evaluadas con el mismo modelo. No cuantifican, en cambio, la variabilidad entre ejecuciones de entrenamiento, puesto que

cada variante se ha ajustado una sola vez. Una réplica con otra semilla podría desplazar las curvas, y esa incertidumbre queda fuera de las cifras reportadas.

La tercera afecta a la selección del modelo. Las 50 condiciones reservadas se evalúan periódicamente y determinan qué punto de control se conserva, de modo que actúan como conjunto de selección y no como evaluación final independiente, tal como precisa el Apartado 3.7. El máximo reportado presenta por ello un sesgo optimista, que la media de las tres últimas evaluaciones ayuda a acotar. Cerrar el problema exigiría evaluar los puntos de control seleccionados sobre un bloque de semillas disjunto del empleado en la selección, medida que no se ha ejecutado y que se propone en el Apartado 5.2.

Por último, la evaluación cada 50 épocas ofrece una visión discreta de la trayectoria y puede omitir máximos intermedios. V2 añade una reanudación, y solo su tiempo acumulado requiere extrapolación parcial. Estas restricciones aconsejan interpretar las cifras como evidencia del experimento realizado, no como una ordenación universal de los codificadores.

5. CONCLUSIONES

Este capítulo interpreta los resultados del Capítulo 4 a la luz de los objetivos y las hipótesis enunciados en los capítulos anteriores. En primer lugar, formula una conclusión por cada objetivo específico y precisa qué permiten afirmar los datos. A continuación, expone las líneas de continuación abiertas por el estudio.

Las cinco variantes disponen de resultados, aunque sus presupuestos de entrenamiento no son uniformes. Esta diferencia limita la comparación temporal y se incorpora a la interpretación de las puntuaciones.

5.1 Conclusiones del trabajo

Sobre la implementación de las variantes

El primer objetivo específico se cumple. Las cinco variantes del codificador visual funcionan dentro de la misma *Diffusion Policy*, entregan un vector de condicionamiento de idéntica dimensión y comparten red generativa y protocolo de evaluación. Las diferencias de presupuesto y tamaño de lote quedan documentadas, por lo que la comparación se acota al bloque visual con esas dos salvedades.

Asimismo, se han evaluado las cinco configuraciones previstas. V3 se interrumpió tras 155 épocas de un presupuesto de 300; las demás finalizaron por parada anticipada o por agotamiento del presupuesto asignado. El estudio responde así a las dos preguntas del Apartado 1.2, con las limitaciones causales descritas más adelante.

Sobre el rendimiento de las variantes

La conclusión principal contradice la hipótesis de partida: ningún bloque visual preentrenado mejora el rendimiento en Push-T. La línea base alcanza 0,864, frente a 0,668 en V1, 0,648 en V2, 0,622 en V3 y 0,535 en V4. Las diferencias respecto a V0 abarcan entre 0,197 y 0,329 puntos. Las cuatro conservan significación tras corregir los diez contrastes mediante el procedimiento de Holm, con valores ajustados iguales o inferiores a 0,0030.

Asimismo, la segunda hipótesis tampoco se sostiene. El ajuste fino del codificador preentrenado no supera a su congelación, puesto que V2 se queda 0,020 por debajo de V1. Esa diferencia no alcanza significación ($p_{\text{Holm}} = 1,00$) y su intervalo contiene el cero. Por tanto, no se demuestra que una estrategia supere a la otra. La incertidumbre entre semillas de entrenamiento, no cuantificada, se suma a esa indeterminación.

El ajuste fino sí presenta una dinámica de entrenamiento menos favorable. V2 permanece estancada alrededor de 0,11 hasta la época 50, cuando V0 y V1 ya rebasan 0,53. Además, sobreajusta antes y con mayor intensidad: en la época 250 su pérdida de validación alcanza 0,380, frente a 0,120 de V1 y 0,077 de V0. En conjunto, la actualización de los pesos degrada la representación de partida sin construir otra más útil con las demostraciones disponibles.

Los transformadores congelados tampoco revierten el resultado. DINOv2 ViT-S/14 supera a CLIP ViT-B/16 por 0,087 puntos, pero la diferencia no es significativa ($p_{\text{Holm}} = 0,5532$). Tras

la corrección familiar, ninguna de las seis comparaciones entre variantes preentrenadas establece un orden significativo. La familia del codificador no explica por sí sola las diferencias observadas.

Una explicación plausible atiende a la distancia entre dominios. Push-T contiene imágenes sintéticas de fondo uniforme y pocas formas, mientras que ImageNet representa categorías de objetos naturales [16]. DINOv2 y CLIP amplían los datos y los objetivos de preentrenamiento, pero sus descriptores globales tampoco mejoran la política. El resultado es coherente con dos antecedentes: R3M tampoco supera al codificador entrenado de extremo a extremo en Push-T [20], [3], y el estudio con DINOv3 concluye que la transferencia depende de la tarea y de la estrategia de adaptación [7].

La atribución causal exige una precisión. V0 difiere de V1 y V2 en cuatro factores: la inicialización, la resolución de entrada, el recorte aleatorio y la agregación espacial. Frente a V3 y V4 cambia, además, la arquitectura. V0 conserva la localización mediante un *spatial softmax* [25], mientras que las variantes preentrenadas entregan un descriptor global. En una tarea definida por posiciones relativas, esta diferencia puede ser tan relevante como la inicialización.

Las conclusiones anteriores se refieren, por tanto, al codificador junto con su cadena de preprocesado, entendidos como un bloque, y no a la inicialización aislada. Separar los cuatro efectos requiere entrenamientos adicionales, descritos en el Apartado 5.2.

Sobre el coste computacional

La línea base obtiene la mejor puntuación y el menor coste unitario. Cada época de V0 ocupa 1,6 min, frente a 5,3 min de V1, 14,7 min de V2, 2,3 min de V3 y 4,0 min de V4. Las variantes preentrenadas son, por tanto, peores y entre 1,5 y 9,5 veces más costosas por época. El tiempo total conjunto asciende a 237,1 h, aunque no sirve para ordenar variantes porque los presupuestos fueron distintos.

El sobrecoste no depende del número de parámetros entrenables. V1, V3 y V4 entrenan 277 millones, frente a los 288 millones de V0, y aun así requieren más tiempo. La resolución, la arquitectura y el tamaño de lote también intervienen. De ahí se sigue una conclusión práctica: congelar el codificador no garantiza un entrenamiento más barato.

El tercer objetivo específico se cumple con los cuatro indicadores anunciados. A los parámetros y los tiempos de entrenamiento se añaden la latencia de inferencia y el pico de memoria gráfica, ambos medidos bajo un protocolo común a las cinco variantes.

De esas dos medidas se siguen dos conclusiones que el tiempo por época no permitía alcanzar. La primera es que el codificador apenas influye en la latencia de la política. Aislado cuesta entre 2,7 ms y 16,1 ms según la variante, pero la llamada completa varía solo un 1,5 %, porque los 100 pasos de difusión dominan el tiempo. Ninguna variante alcanza el control en tiempo real sobre este equipo, y la vía para conseguirlo pasa por el muestreo y no por el bloque visual.

La segunda es que el ajuste fino, y no el preentrenamiento, es lo que agota la memoria. V2 reserva 10,293 gibibytes sobre una tarjeta de 8 y solo se completa gracias al respaldo en memoria del sistema, lo que explica su coste por época. Las cuatro variantes restantes se mantienen entre el 78 % y el 90 % de la capacidad. En inferencia ninguna supera el 23 %: la restricción de memoria de este trabajo pertenece al entrenamiento, no al despliegue.

Respuesta al objetivo general

En conjunto, la respuesta al objetivo general es inequívoca para la configuración estudiada. Con 206 demostraciones de Push-T, la ResNet-18 entrenada de extremo a extremo a resolución nativa domina en rendimiento y coste a las cuatro alternativas preentrenadas. Ni DINOv2 [18] ni CLIP [19] alteran esa conclusión. La ventaja corresponde al bloque visual completo y no permite atribuir el efecto exclusivamente al preentrenamiento.

5.2 Trabajo futuro

Del estudio se derivan cinco líneas de continuación, ordenadas por su capacidad para afianzar o corregir las conclusiones anteriores.

En primer lugar, aislar los cuatro factores confundidos entre la línea base y las variantes preentrenadas. Un entrenamiento adicional de ResNet-18 desde cero a 224 píxeles y sin recorte aleatorio separa el efecto de la inicialización del efecto del preprocesado. Su coste por época coincide con el de V2, del orden de 15 min, puesto que el codificador se actualiza en ambos casos y la resolución de entrada es la misma; bajo el criterio de parada, el total se sitúa en torno a las 85 h. Una segunda ejecución desde cero a 96 píxeles y sin recorte aleatorio aislaría el efecto del aumento de datos por unas 17 h, coste equivalente al de V0.

En segundo lugar, repetir las variantes seleccionadas con las semillas 43 y 44. Los contrastes del Capítulo 4 cubren la dispersión entre condiciones iniciales, pero no la variabilidad entre ejecuciones de entrenamiento; solo la réplica permite estimarla, limitación declarada desde el Apartado 1.4 y cuantificada en el Apartado 3.9.

En tercer lugar, evaluar los puntos de control seleccionados sobre un bloque de semillas de entorno disjunto del empleado para seleccionarlos. Ese paso convertiría la medida actual, que actúa como puntuación de selección, en una evaluación final independiente, y su coste se reduce a tres ejecuciones de inferencia.

En cuarto lugar, examinar la agregación espacial a la salida de los codificadores preentrenados. La línea base conserva la localización de las activaciones mediante un *spatial softmax* [25],

si una agregación que preserve la geometría recupera parte de la diferencia observada.

Por último, extender el protocolo a otras tareas y a un robot físico. Push-T aporta un banco de pruebas controlado, pero sus imágenes sintéticas son precisamente el escenario menos favorable para un codificador preentrenado sobre fotografías naturales. La conclusión podría invertirse en observaciones reales, extremo que este trabajo no puede resolver.

6. PRESUPUESTO

Este capítulo cuantifica el coste de ejecución del proyecto. El presupuesto recoge cuatro partidas: material fungible y suministros, amortización de los equipos, licencias de software y mano de obra. A ellas se añaden unos costes indirectos y, con todo, se obtiene el importe total.

Tres criterios de imputación gobiernan las cifras que siguen. En primer lugar, el periodo imputado abarca siete meses, de marzo a septiembre de 2026, que comprenden desde la revisión bibliográfica hasta la entrega de la memoria. En segundo lugar, los equipos se amortizan de forma lineal y se imputa la fracción correspondiente a esos siete meses, con independencia de su uso efectivo dentro de cada jornada. En tercer lugar, los importes se expresan sin impuestos indirectos, puesto que el trabajo se desarrolla en el ámbito académico y no da lugar a facturación.

6.1 Material fungible y suministros

El proyecto es de naturaleza computacional, de modo que apenas consume material físico. La única partida relevante es la energía eléctrica, dominada por los entrenamientos desatendidos. Según el Apartado 3.9, estos ocuparon 237,1 h de cómputo; con un consumo aproximado de 150 W bajo carga, suman unos 36 kilovatios-hora. El resto de la dedicación añade otros 24 kilovatios-hora, con el equipo en régimen de trabajo ordinario. La Tabla 12 recoge ambos conceptos.

Tabla 12. Presupuesto de material fungible y suministros.

Concepto	Cantidad	Coste unitario (€)	Total (€)
Energía eléctrica (kWh)	60	0,15	9,00
Material de oficina e impresión	1	45,00	45,00
Total			54,00

Se adopta un precio de 0,15 €/kWh, valor de referencia para el suministro doméstico durante el periodo imputado.

Fuente: elaboración propia.

6.2 Amortización de los equipos

El proyecto se ejecuta sobre un único equipo, descrito en el Apartado 3.8. Dado que su vida útil excede la duración del trabajo, no se imputa el precio de adquisición sino la amortización correspondiente al periodo de uso, según la Ecuación (6).

$$A = C \cdot \frac{t}{V} \quad (6)$$

En la Ecuación (6), A es el importe imputado, C el precio de adquisición, t el periodo de uso y V la vida útil, ambos expresados en meses. Se aplica amortización lineal y sin valor

residual, criterio habitual para equipos de proceso de información. La Tabla 13 detalla el resultado para cada elemento.

Tabla 13. Amortización de los equipos imputada al proyecto.

Equipo	Precio (€)	Vida útil (años)	Uso (meses)	Imputación (€)
Portátil con GPU dedicada	1.800,00	4	7	262,50
Monitor externo	180,00	5	7	21,00
Unidad externa de respaldo	60,00	3	7	11,67
Total				295,17

Fuente: elaboración propia.

Conviene señalar que la unidad de procesamiento gráfico condiciona el diseño experimental, según se argumentó en el Apartado 3.8, pero apenas condiciona el presupuesto. Su amortización forma parte de los 262,50 € imputados al portátil, importe menor que el coste de dos jornadas de trabajo.

6.3 Licencias de software

Todas las herramientas empleadas son de código abierto o de uso gratuito, de modo que esta partida asciende a 0,00 €. La Tabla 6 enumera la pila de software: Python, PyTorch, torchvision, timm, diffusers, robomimic, Hydra y Zarr se distribuyen bajo licencias libres. A ellas se suman la distribución Ubuntu, el sistema de composición T_EX Live y el control de versiones Git, también libres. La licencia de Windows 11 viene incluida en el precio del portátil y queda, por tanto, recogida en el Apartado 6.2.

6.4 Mano de obra

La mano de obra distingue dos perfiles. El autor asume la implementación, la ejecución, el análisis y la redacción, con el coste horario de un ingeniero de datos junior. El director asume la definición del alcance, el seguimiento periódico y la revisión de la memoria, con el coste horario de un investigador sénior. Ambos importes corresponden al coste bruto para la entidad, incluidas las cargas sociales.

La Tabla 14 desglosa la dedicación del autor por fases. El reparto refleja el peso real de cada actividad: la preparación del entorno de cómputo resultó más costosa de lo previsto, mientras que la ejecución de los entrenamientos, aunque ocupó más de 237 horas de máquina, exigió poca intervención por tratarse de procesos desatendidos.

La dedicación del director se estima en 40 horas: unas 20 reuniones de seguimiento de una hora, repartidas a lo largo de los siete meses, y otras 20 horas de revisión de los borradores. La Tabla 15 aplica los costes horarios a ambos perfiles.

6.5 Resumen del presupuesto

La Tabla 16 agrega las cuatro partidas anteriores y añade unos costes indirectos del 15 % sobre el subtotal, porcentaje que cubre espacio de trabajo, conectividad y servicios generales no imputables a un concepto concreto.

Tabla 14. Dedicación del autor por fases del proyecto.

Fase	Horas
Revisión bibliográfica y estado del arte	70
Preparación del entorno de cómputo	55
Implementación de las variantes del codificador	90
Ejecución y supervisión de los entrenamientos	60
Análisis de resultados y comprobación cualitativa	65
Redacción de la memoria	110
Total	450

Fuente: elaboración propia.

Tabla 15. Presupuesto de mano de obra.

Perfil	Horas	Coste horario (€)	Total (€)
Ingeniero de datos junior (autor)	450	30,00	13.500,00
Investigador sénior (director)	40	60,00	2.400,00
Total	490		15.900,00

Fuente: elaboración propia.

Tabla 16. Resumen del presupuesto.

Partida	Importe (€)
Material fungible y suministros	54,00
Amortización de equipos	295,17
Licencias de software	0,00
Mano de obra	15.900,00
Subtotal	16.249,17
Costes indirectos (15 %)	2.437,38
Total	18.686,55

Fuente: elaboración propia.

El coste total del proyecto asciende, por tanto, a 18.686,55 €. La distribución entre partidas resulta muy desigual: la mano de obra representa el 85 % del total, mientras que los equipos apenas alcanzan el 1,6 %. Esa asimetría es característica de un proyecto de investigación computacional ejecutado sobre hardware propio.

De ahí se sigue una consideración económica sobre el diseño experimental. Las limitaciones descritas en el Apartado 3.9 no proceden del presupuesto, sino del tiempo de calendario: ampliar el estudio a tres semillas por variante multiplicaría por tres el cómputo sin apenas alterar el coste, ya que la amortización y la energía son partidas menores. El factor limitante es que esas horas de máquina deben transcurrir dentro del plazo del proyecto, no lo que cuestan.

REFERENCIAS

- [1] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, ser. JMLR: W&CP, vol. 15, Fort Lauderdale, FL, USA, 2011, pp. 627–635.
- [2] P. Florence, C. Lynch, A. Zeng, O. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, “Implicit behavioral cloning,” in *Proc. 5th Conf. Robot Learning (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 164, London, U.K., 2021, pp. 158–168.
- [3] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10–11, pp. 1684–1704, 2025, doi: 10.1177/02783649241273668.
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, Vancouver, Canada, 2020, pp. 6840–6851.
- [5] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations,” arXiv:2403.03954, 2024, presentado posteriormente en Robotics: Science and Systems (RSS).
- [6] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg, “Consistency policy: Accelerated visuomotor policies via consistency distillation,” arXiv:2405.07503, 2024.
- [7] T. Egbe, P. Wang, Z. Guo, and Z. Chen, “DINOv3-Diffusion Policy: Self-supervised large visual model for visuomotor diffusion policy learning,” arXiv:2509.17684, 2025.
- [8] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Proc. 5th Conf. Robot Learning (CoRL)*, London, U.K., 2021, arXiv:2108.03298.
- [9] Y. Song and S. Ermon, “Generative modeling by estimating gradients of the data distribution,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, Vancouver, Canada, 2019, pp. 11 895–11 907.
- [10] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021, arXiv:2011.13456.
- [11] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021, arXiv:2010.02502.
- [12] X. Ma, S. Patidar, I. Haughton, and S. James, “Hierarchical diffusion policy for kinematics-aware multi-task robotic manipulation,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024, arXiv:2403.03890.

- [13] X. Sun, F. Fan, Y. Chen, and D. Rakita, “Optimizing active perception for learning simultaneous viewpoint selection and manipulation with diffusion policy,” arXiv:2409.14615, 2024.
- [14] C. Tie, Y. Chen, R. Wu, B. Dong, Z. Li, C. Gao, and H. Dong, “ET-SEED: Efficient trajectory-level SE(3) equivariant diffusion policy,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025, arXiv:2411.03990.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.
- [16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Miami, FL, USA, 2009, pp. 248–255.
- [17] A. Dosovitskiy, L. Beyer, A. Kolesnikov *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021, arXiv:2010.11929.
- [18] M. Oquab, T. Darcet, T. Moutakanni *et al.*, “DINOv2: Learning robust visual features without supervision,” *Transactions on Machine Learning Research*, 2024, arXiv:2304.07193.
- [19] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Proc. 38th Int. Conf. Machine Learning (ICML)*, ser. Proc. Mach. Learn. Res., vol. 139, 2021, pp. 8748–8763.
- [20] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, “R3M: A universal visual representation for robot manipulation,” in *Proc. 6th Conf. Robot Learning (CoRL)*, 2022, arXiv:2203.12601.
- [21] T. Xiao, I. Radosavovic, T. Darrell, and J. Malik, “Masked visual pre-training for motor control,” arXiv:2203.06173, 2022.
- [22] S. Karamcheti, S. Nair, A. S. Chen, T. Kollar, C. Finn, D. Sadigh, and P. Liang, “Language-driven representation learning for robotics,” in *Proc. Robotics: Science and Systems (RSS)*, Daegu, Republic of Korea, 2023, doi: 10.15607/RSS.2023.XIX.032.
- [23] A. Majumdar, K. Yadav, S. Arnaud, J. Ma, C. Chen, S. Silwal, A. Jain, V.-P. Berges, T. Wu, J. Vakil, P. Abbeel, J. Malik, D. Batra, Y. Lin, O. Maksymets, A. Rajeswaran, and F. Meier, “Where are we in the search for an artificial visual cortex for embodied intelligence?” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 655–677, doi: 10.52202/075280-0031.
- [24] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proc. AAAI Conf. Artificial Intelligence*, New Orleans, LA, USA, 2018, arXiv:1709.07871.
- [25] S. Levine, C. Finn, T. Darrell, and P. Abbeel, “End-to-end training of deep visuomotor policies,” *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016.
- [26] Y. Wu and K. He, “Group normalization,” in *Proc. European Conf. Computer Vision (ECCV)*, Munich, Germany, 2018, pp. 3–19.

- [27] R. Wightman, “PyTorch image models,” Repositorio de software, <https://github.com/huggingface/pytorch-image-models>, 2019.
- [28] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, New Orleans, LA, USA, 2019, arXiv:1711.05101.
- [29] A. Paszke, S. Gross, F. Massa *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, Vancouver, Canada, 2019, pp. 8024–8035.

BIBLIOGRAFÍA

M. Britez, "Guía de estudio técnica, matemática y conceptual: Diffusion Policy," Monografía autocontenida sobre Chi *et al.*, "Diffusion Policy: Visuomotor policy learning via action diffusion", 2026.

A. TÍTULO DEL PRIMER ANEXO

[Material de apoyo: configuraciones completas, tablas extensas, listados de código. Todo anexo se menciona al menos una vez en el cuerpo de la memoria.]

```
1 def ejemplo():  
2     return None
```

Código A.1. Fragmento de código de ejemplo.

B. TÍTULO DEL SEGUNDO ANEXO

[Contenido.]